Melanie Giovanna Valle Benavides – 202625965

Laboratorio 3

- **Pregunta 1:** REQ1 y REQ2 pueden entrar en tensión en picos de demanda. En el contexto de Cheapest, si una decisión de arquitectura mejora el p99 pero empeora el porcentaje de éxito, ¿qué criterio de priorización usaría y por qué? Responda considerando impacto al tendero e impacto al negocio.

Priorizaría el porcentaje de éxito de pedidos por encima del p99 en momentos de pico.

Un pedido que falla es una venta que Cheapest pierde directamente y un tendero que quedó sin su mercancía, eso es daño irreversible. Una respuesta lenta (p99 alto) es incómoda, pero el tendero igual recibe su resultado y puede esperar un poco más, especialmente si sabe que es un evento de alta demanda. En otras palabras: es peor decirle "no" a un tendero que decirle "espera un poco".

Para el negocio es igual: perder pedidos en un pico de promociones significa perder la oportunidad de venta más grande del período, mientras que la lentitud rara vez ahuyenta al tendero de inmediato, solo genera frustración temporal.

- **Pregunta 2:** La topología de este laboratorio combina monolito + ALB + múltiples instancias. ¿Qué atributo de calidad está mejorando realmente y cuál podría estar sobreestimándose si no se analizan correctamente las métricas? Qué criterio usaría para evitar conclusiones engañosas cuando hay balanceo de carga pero no hay desacoplamiento funcional interno.

El ALB mejora principalmente la disponibilidad (si una instancia se cae, las otras dos siguen atendiendo) y algo de capacidad total (más máquinas procesando en paralelo).

Lo que se puede sobreestimar es la escalabilidad real del sistema. Como las 3 instancias comparten la misma base de datos, si el cuello de botella está ahí (que es lo más probable en un monolito), agregar más instancias de app no va a mejorar el throughput más allá de cierto punto, solo se ve bien en el papel porque "hay más servidores".

Para no llegar a conclusiones engañosas, el criterio es: medir la base de datos por separado (CPU, conexiones activas, locks) durante la prueba, no solo mirar el throughput total del ALB. Si el throughput deja de subir aunque agregues instancias de app, la prueba lo confirma: el límite está en la base de datos, no en la capa de aplicación, y el balanceo de carga no estaba realmente "escalando" el sistema, solo estaba repartiendo trabajo hacia el mismo cuello de botella.

- **Pregunta 3:** Antes de desplegar, justifique por qué en Cheapest tendría sentido separar App y DB en instancias distintas incluso si el costo y la complejidad operativa aumentan.

¿Qué riesgos de arquitectura está mitigando y cuáles está introduciendo al mover un monolito local a una topología distribuida en AWS? Represente su respuesta con una tabla comparativa donde se vean explícitamente los riesgos mitigados e introducidos.

Riesgo	Antes (todo junto)	Ahora (App y DB separadas)
Mitigado: un fallo tumba todo	Si el proceso de la app se cae o consume toda la RAM, se lleva la base de datos con él	Un fallo en una instancia de app no afecta la base de datos, y viceversa
Mitigado: escalar sin control	No se puede escalar solo la app sin también duplicar la base de datos	Se puede agregar más instancias de app sin tocar la base de datos
Mitigado: seguridad mezclada	Cualquier acceso a la máquina expone tanto la app como los datos	Cada máquina tiene su propio Security Group con permisos distintos
Introducido: latencia de red	Las consultas eran locales (mismo proceso)	Ahora cada consulta viaja por red entre instancias, aunque sea rápida
Introducido: complejidad operativa	Una sola máquina que administrar	Hay que gestionar y monitorear 4 máquinas, su red y sus fallos por separado
Introducido: punto único en la DB	No aplica	Ahora todas las instancias de app dependen de una sola instancia de base de datos; si esa falla, cae todo el sistema

Vale la pena porque, aunque se introduce complejidad, el diseño anterior no podía crecer ni sobrevivir a un fallo parcial, para un sistema que Cheapest necesita disponible durante picos de promociones, esas dos cosas son más valiosas que la simplicidad operativa de tener todo en una sola máquina.

- **Pregunta 4:** Proponga un diseño mínimo de seguridad para Cheapest que elimine la exposición pública de infraestructura que debería ser privada. Debe incluir al menos: origen permitido de tráfico, estrategia de segmentación de red

El problema actual es que PostgreSQL (5432) está abierto a todo internet (0.0.0.0/0), cuando en realidad solo las 3 instancias de app necesitan hablarle.

Origen permitido de tráfico:

- Puerto 5432 (DB): permitir solo desde el Security Group de las instancias de app (Cheapest-http), no desde 0.0.0.0/0.

- Puerto 3000 (app): en vez de abrirlo a todo internet, permitir solo desde el Security Group del ALB (Cheapest-alb), nadie debería poder pegarle a la app directo, solo a través del balanceador.
- Puerto 22 (SSH): restringir a la IP de quien administra (tu IP, no Anywhere), o mejor, usar EC2 Instance Connect y eliminar la regla abierta.

Segmentación de red:

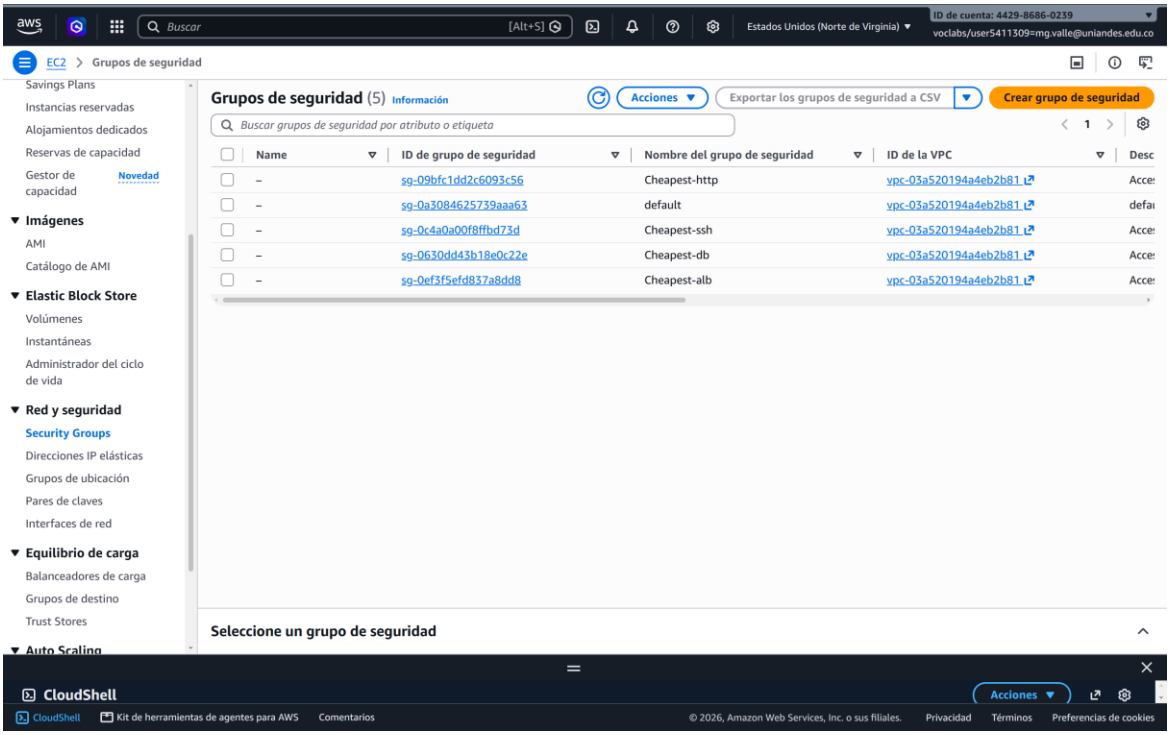
- Poner la base de datos en una subred privada (sin IP pública ni ruta a internet), ya que nunca necesita ser accedida desde afuera.
- Dejar las instancias de app en subred pública solo porque necesitan recibir tráfico del ALB (que sí está en la subred pública).
- Esto crea capas: internet → ALB (público) → apps (público, pero solo alcanzable por el ALB) → base de datos (privada, solo alcanzable por las apps).

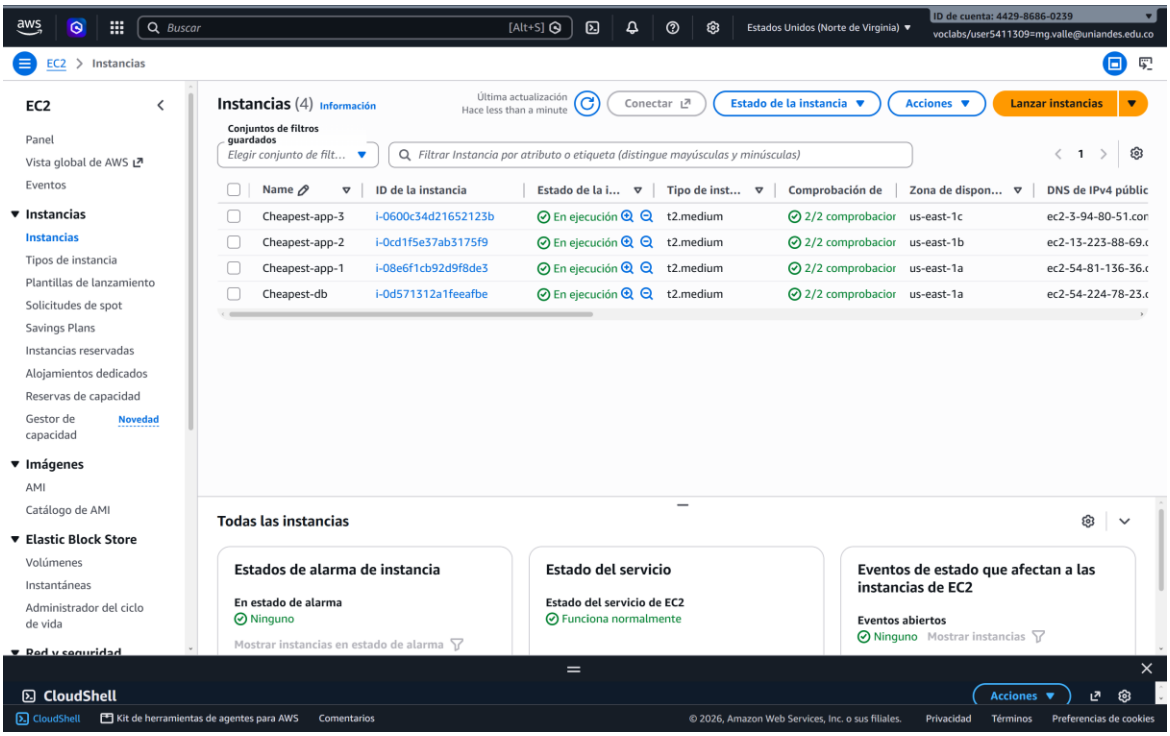
- **Pregunta 5:** Para el laboratorio usaremos la IP privada para conectar la aplicación a la base de datos. Compare el uso de IP privada vs IP pública para la conexión entre App y Base de Datos en este laboratorio. ¿Qué diferencias esperaría en seguridad, latencia, estabilidad y costo operativo para Cheapest, y en qué casos podría justificarse usar cada opción? Apoye la comparación con una tabla.

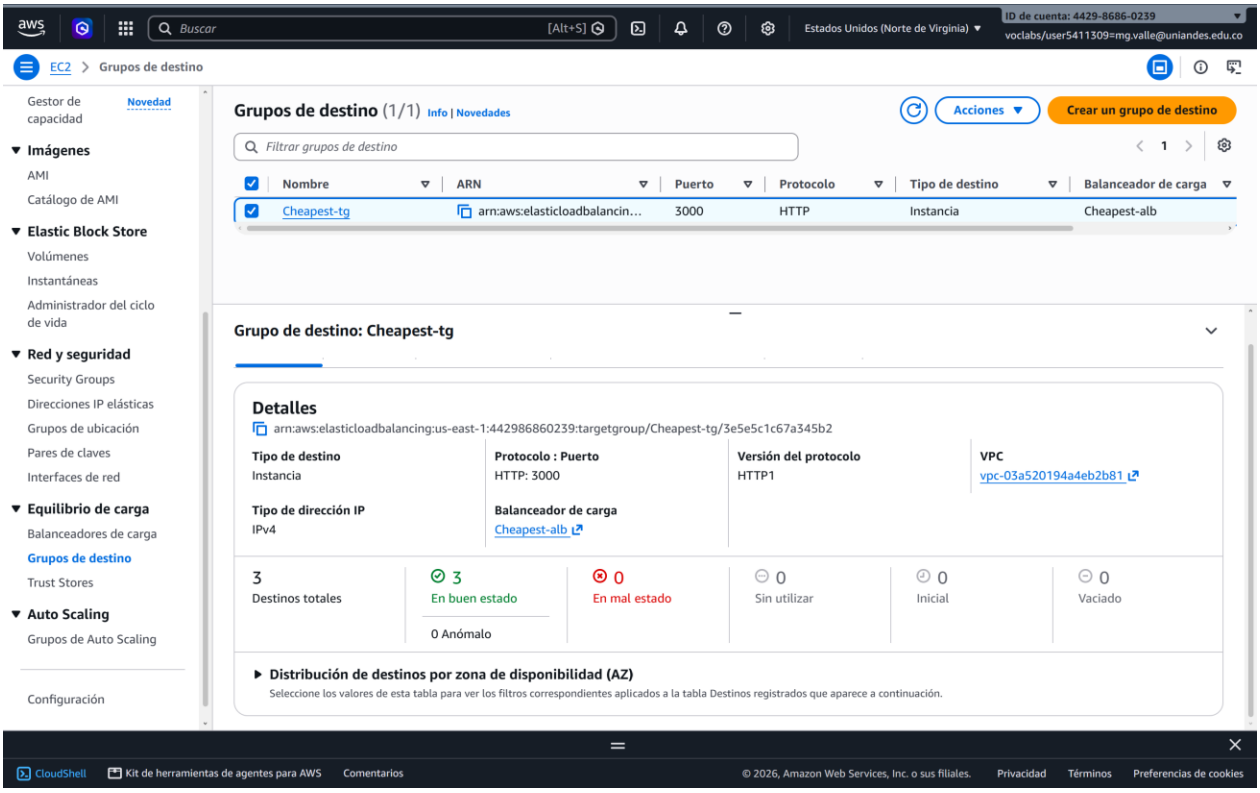
Aspecto	**IP privada**	**IP pública**
Seguridad	El tráfico nunca sale a internet, mucho más difícil de interceptar o atacar desde afuera	Expuesto a internet; cualquier fuga de configuración (como el Security Group abierto) se vuelve explotable globalmente
Latencia	Menor, tráfico dentro de la misma red de AWS	Mayor, el tráfico da una vuelta más larga aunque las máquinas estén físicamente cerca
Estabilidad	Más estable, no depende de rutas externas de internet	Puede verse afectada por congestión o cambios en la IP pública si la instancia se reinicia
Costo	Gratis (tráfico interno de AWS)	AWS cobra por transferencia de datos saliente/entrante por IP pública

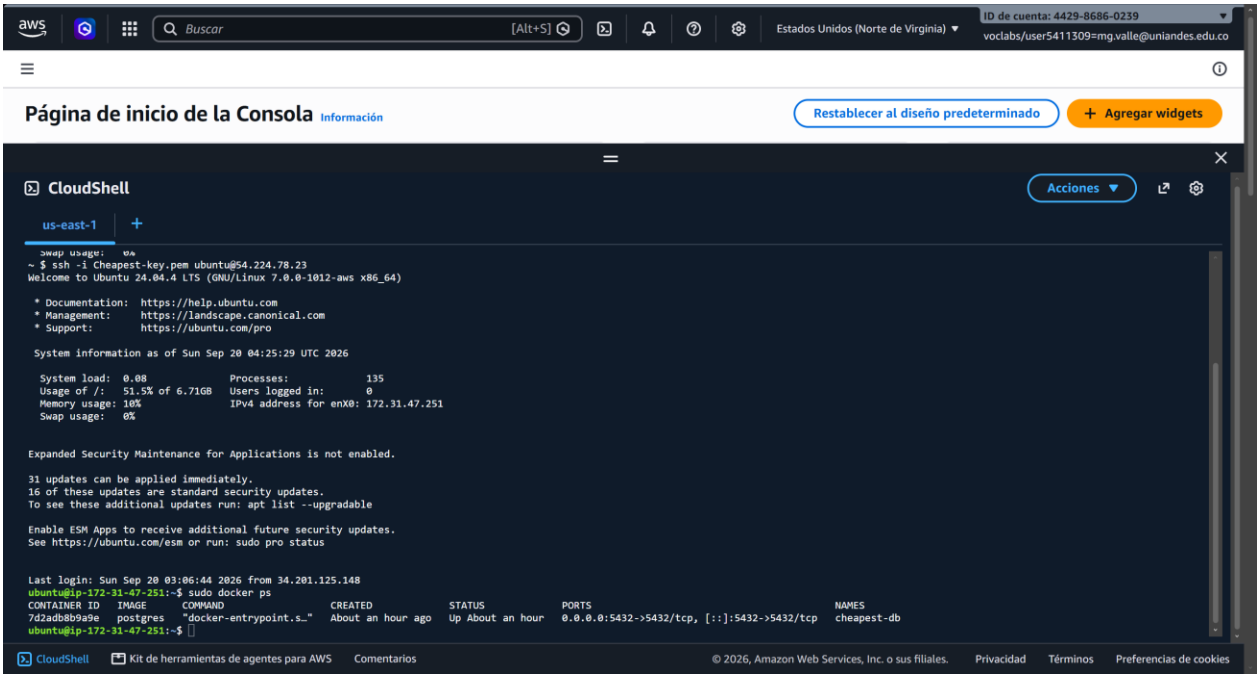
Para Cheapest, usar IP privada es la opción correcta siempre que App y DB estén en la misma VPC, que es justo el caso. La IP pública para esta conexión solo se justificaría si la base de datos estuviera en un proveedor distinto o fuera de la VPC, algo que no aplica aquí y que, de hecho, sería una mala práctica evitable.

Evidencia de Despliegue






Buscar
[Alt+S]
Estados Unidos (Norte de Virginia)
ID de cuenta: 4429-8686-0239
voclabs/user5411309=mg.valle@uniandes.edu.co
EC2 > Grupos de destino
Gestor de capacidad
Novedad
Imágenes
AMI
Catálogo de AMI
Elastic Block Store
Volúmenes
Instantáneas
Administrador del ciclo de vida
Red y seguridad
Security Groups
Direcciones IP elásticas
Grupos de ubicación
Pares de claves
Interfaces de red
Equilibrio de carga
Balanceadores de carga
Grupos de destino
Trust Stores
Auto Scaling
Grupos de Auto Scaling
Configuración
Grupos de destino (1/1) Info | Novedades
Acciones
Crear un grupo de destino
Filtrar grupos de destino
Nombre
ARN
Puerto
Protocolo
Tipo de destino
Balanceador de carga
Cheapest-tg
arn:aws:elasticloadbalancin...
3000
HTTP
Instancia
Cheapest-alb
Grupo de destino: Cheapest-tg
Detalles
arn:aws:elasticloadbalancing:us-east-1:442986860239:targetgroup/Cheapest-tg/3e5e5c1c67a345b2
Tipo de destino
Instancia
Protocolo : Puerto
HTTP: 3000
Versión del protocolo
HTTP1
VPC
vpc-03a520194a4eb2b81
Tipo de dirección IP
IPv4
Balanceador de carga
Cheapest-alb
3
Destinos totales
3
En buen estado
0 Anómalo
0
En mal estado
0
Sin utilizar
0
Inicial
0
Vaciado
Distribución de destinos por zona de disponibilidad (AZ)
Seleccione los valores de esta tabla para ver los filtros correspondientes aplicados a la tabla Destinos registrados que aparece a continuación.
CloudShell
Kit de herramientas de agentes para AWS
Comentarios
© 2026, Amazon Web Services, Inc. o sus filiales.
Privacidad
Términos
Preferencias de cookies


Buscar
[Alt+S]
Estados Unidos (Norte de Virginia)
ID de cuenta: 4429-8686-0239
voclabs/user5411309=mg.valle@uniandes.edu.co
Página de inicio de la Consola Información
Restablecer al diseño predeterminado
+ Agregar widgets
CloudShell
Acciones
us-east-1
~ $ ssh -i Cheapest-key.pem ubuntu@54.224.78.23
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-1012-aws x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
System information as of Sun Sep 20 04:25:29 UTC 2026
System load: 0.08
Usage of /: 51.5% of 6.71GB
Memory usage: 10%
Swap usage: 0%
Processes: 135
Users logged in: 0
IPv4 address for enX0: 172.31.47.251
Expanded Security Maintenance for Applications is not enabled.
31 updates can be applied immediately.
16 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Last login: Sun Sep 20 03:06:44 2026 from 34.201.125.148
ubuntu@ip-172-31-47-251:~$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
7d2adb8b9a9e postgres "docker-entrypoint.s…" About an hour ago Up About an hour 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp cheapest-db
ubuntu@ip-172-31-47-251:~$
CloudShell
Kit de herramientas de agentes para AWS
Comentarios
© 2026, Amazon Web Services, Inc. o sus filiales.
Privacidad
Términos
Preferencias de cookies

PRUEBAS GET

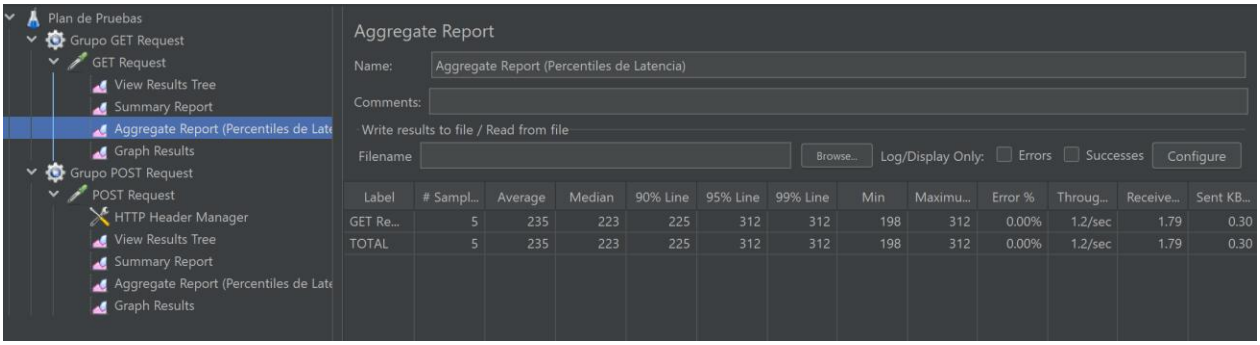
Escenario	Corrida	Threads / users	Ramp-up	Duración	p95 (ms)	p99 (ms)	Throughput	Error %
Smoke test	1	5	5s	—	312	312	1.2/sec	0.00
Smoke test	2	5	5s	—	313	313	1.2/sec	0.00
Smoke test	3	5	5s	—	251	251	1.2/sec	0.00
Smoke test	4	5	5s	—	345	345	1.2/sec	0.00
Baja carga	1	30	10s	—	258	308	3.0/sec	0.00
Baja carga	2	30	10s	—	300	390	3.0/sec	0.00
Baja carga	3	30	10s	—	222	262	3.0/sec	0.00
Baja carga	4	30	10s	—	218	375	3.0/sec	0.00
Carga media	1	100	20s	—	258	381	5.0/sec	0.00
Carga media	2	100	20s	—	238	331	5.0/sec	0.00
Carga media	3	100	20s	—	232	307	5.0/sec	0.00
Carga media	4	100	20s	—	246	275	5.0/sec	0.00
Operación normal	1	450	50s	—	242	1198	9.0/sec	0.00
Operación normal	2	450	50s	—	289	749	9.0/sec	0.00
Operación normal	3	450	50s	—	328	569	9.0/sec	0.00
Operación normal	4	450	50s	—	266	349	9.0/sec	0.00

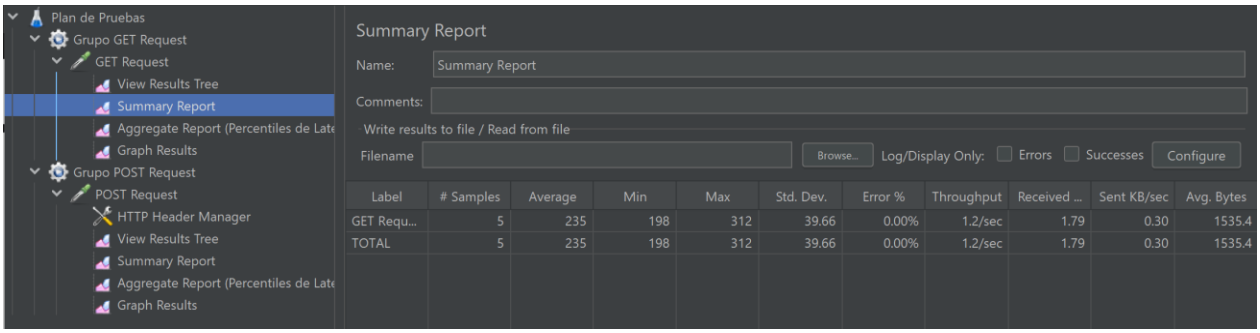
Operación normal	5	450	50s	—	261	303	9.0/sec	0.00
Operación normal	6	450	50s	—	254	466	9.0/sec	0.00
Operación normal	7	450	50s	—	236	274	9.0/sec	0.00
Operación normal	8	450	50s	—	242	278	9.0/sec	0.00
Alta carga	1	1500	75s	60s	32037.87	50203.93	84.20/sec	0.00
Alta carga	2	1500	75s	60s	30972.02	48438.79	82.14/sec	0.00
Alta carga	3	1500	75s	60s	33381.93	52075.20	78.78/sec	0.00
Alta carga	4	1500	75s	60s	33320.42	50510.61	78.89/sec	0.01
Muy alta carga	1	3000	100s	60s	44488.67	69042.22	62.38/sec	0.01
Muy alta carga	2	3000	100s	60s	40766.84	62248.26	64.07/sec	0.01
Muy alta carga	3	3000	100s	60s	47199.45	73149.50	66.04/sec	0.00
Muy alta carga	4	3000	100s	60s	47688.58	71706.87	56.23/sec	0.00
Estrés	1	7500	150s	60s	88353.03	133900.76	47.21/sec	0.00
Estrés	2	7500	150s	60s	84440.52	129206.61	48.54/sec	83.69
Estrés	3	7500	150s	60s	90216.31	133689.87	61.10/sec	100.00
Estrés	4	7500	150s	60s	85181.16	130153.31	57.54/sec	100.00
Estrés fuerte (pico)	1	18000	200s	60s	117887.95	172412.47	75.28/sec	100.00
Estrés fuerte (pico)	2	18000	200s	60s	123171.04	180500.58	75.86/sec	100.00
Estrés fuerte (pico)	3	18000	200s	60s	113233.14	162861.81	74.56/sec	100.00

Estrés fuerte (pico)	4	18000	200s	60s	118009.90	172599.01	66.87/sec	0.00
Estrés fuerte (pico)	5	18000	200s	60s	119093.19	174152.24	69.79/sec	0.00
Estrés fuerte (pico)	6	18000	200s	60s	126799.74	183031.55	61.21/sec	0.00
Estrés fuerte (pico)	7	18000	200s	60s	112211.31	163762.87	57.70/sec	0.01
Estrés fuerte (pico)	8	18000	200s	60s	118181.65	174439.65	60.70/sec	0.00

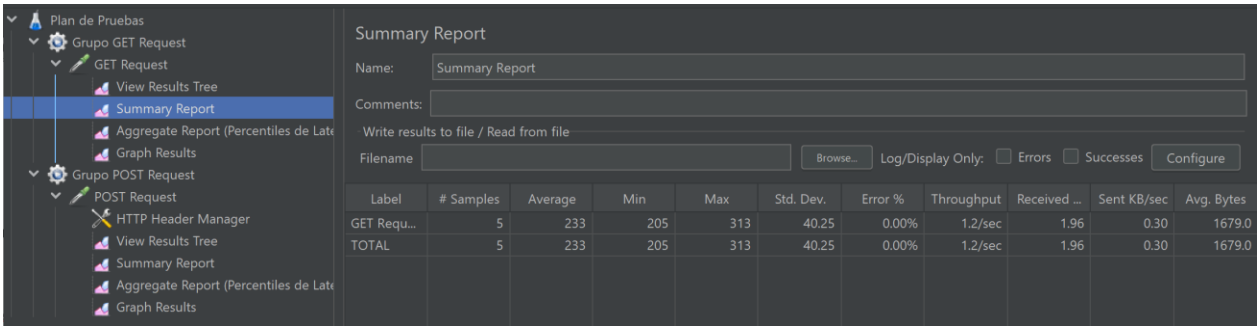
Evidencia

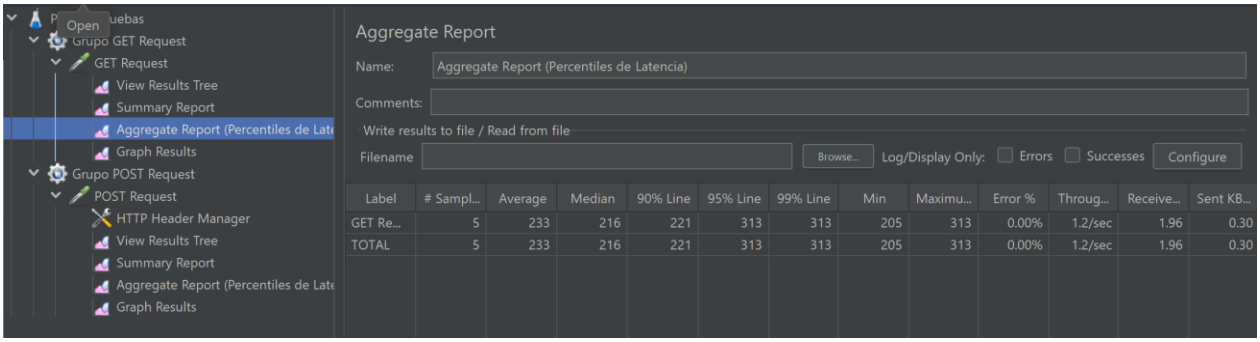
SmokeTest 1



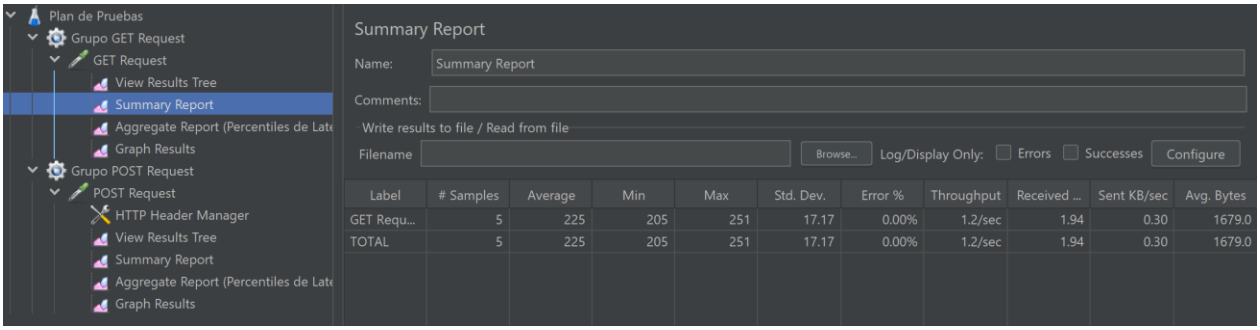


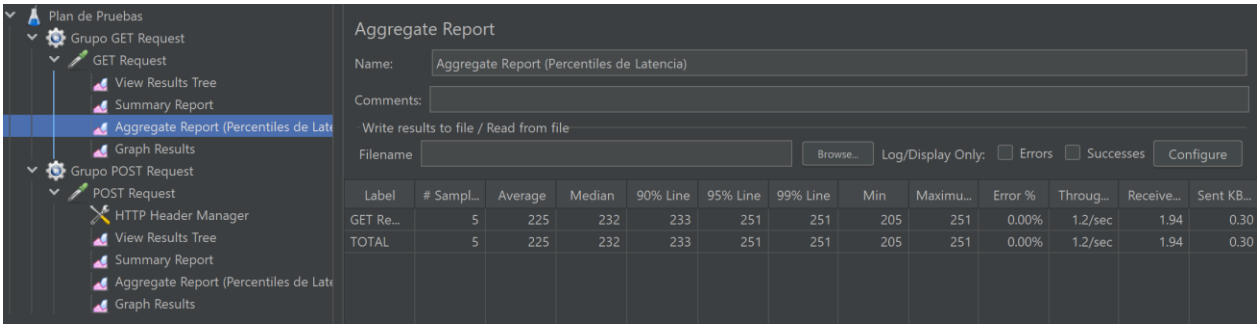
SmokeTest 2



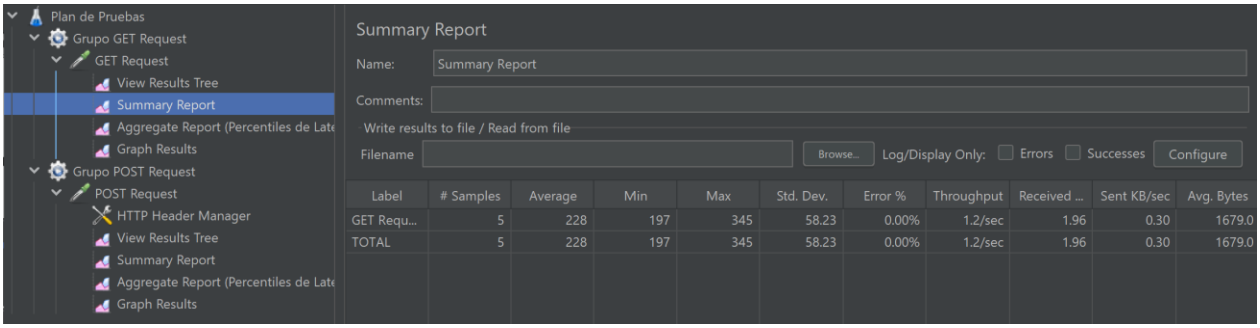


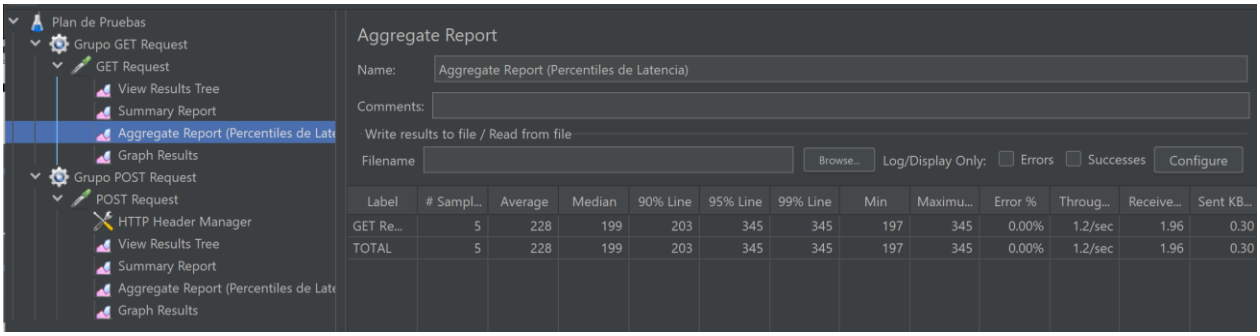
SmokeTest 3



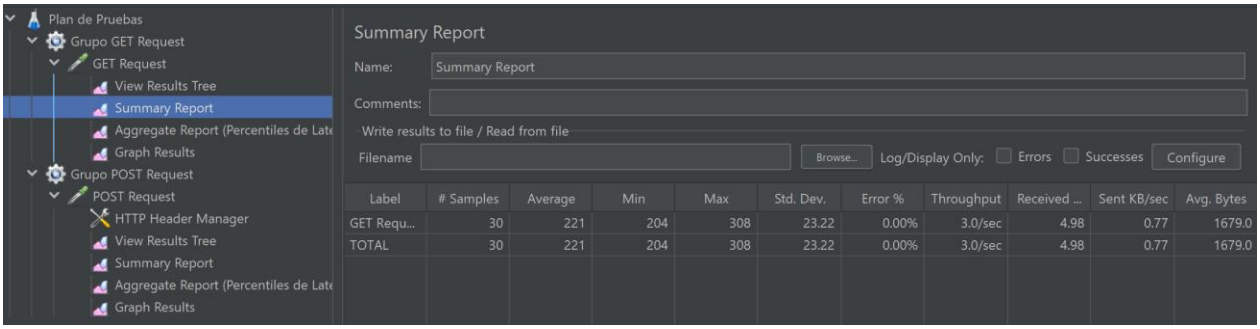


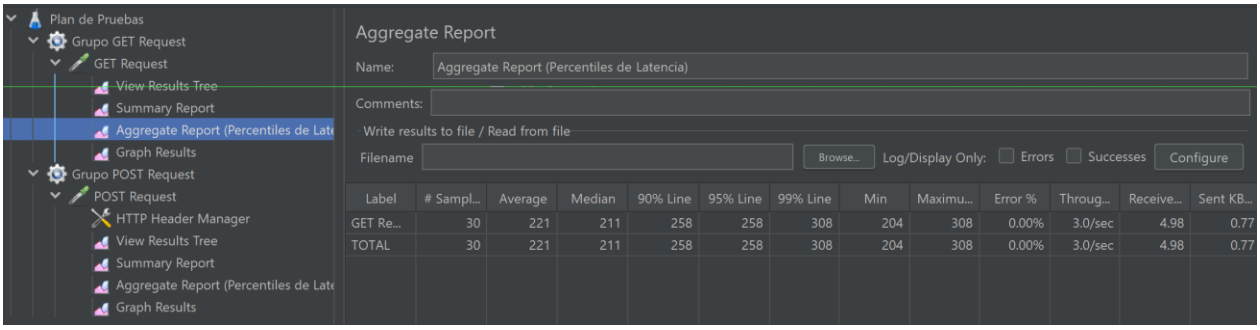
SmokeTest 4



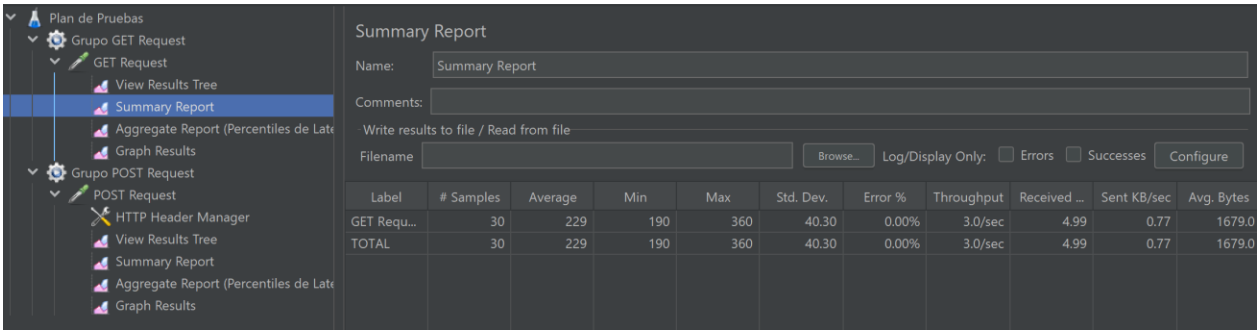


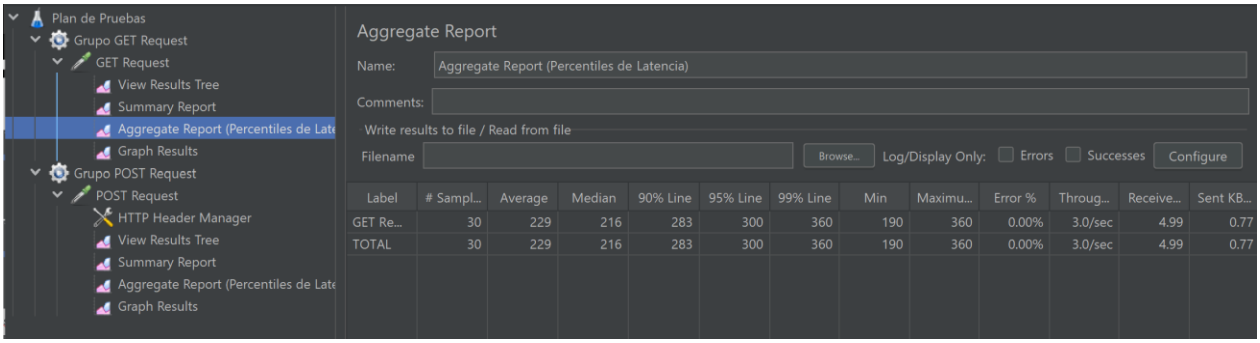
Baja Carga 1



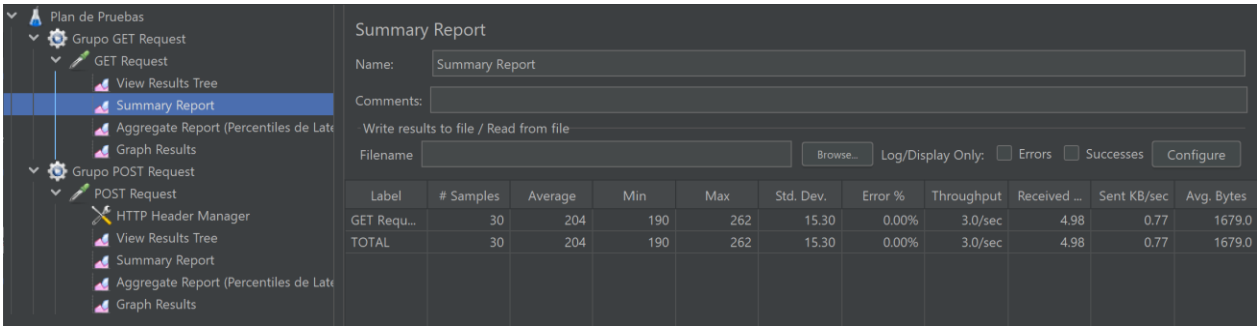


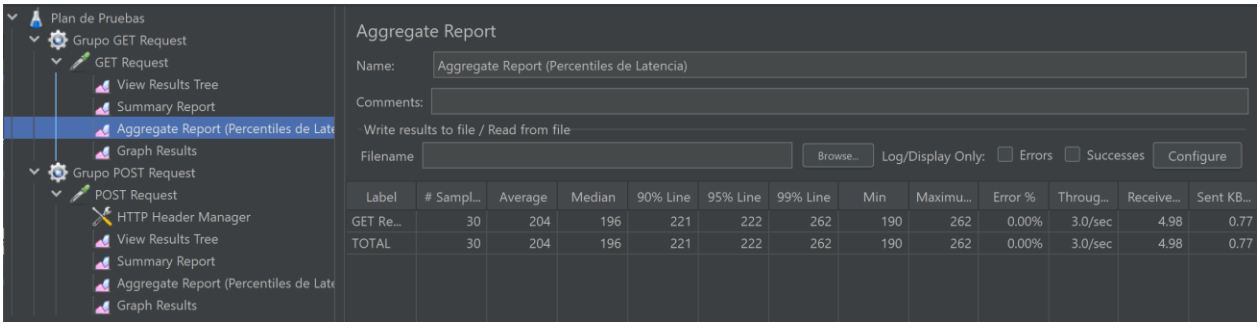
Baje Carga 2



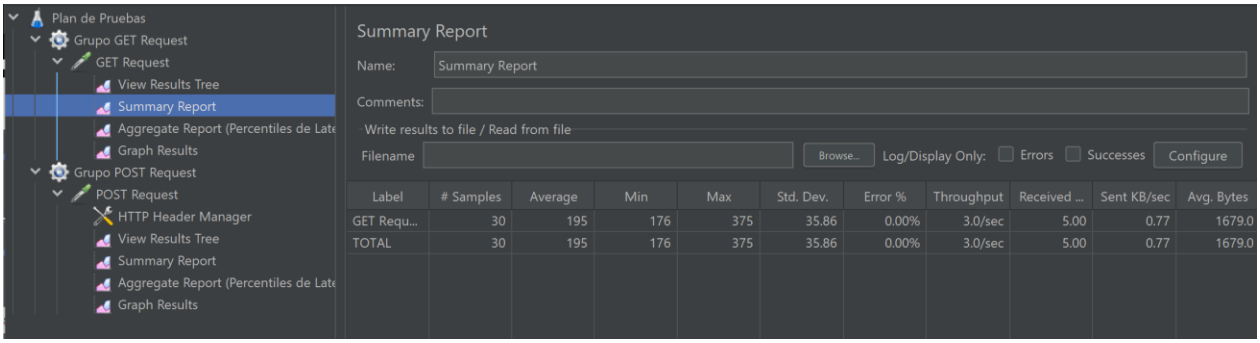


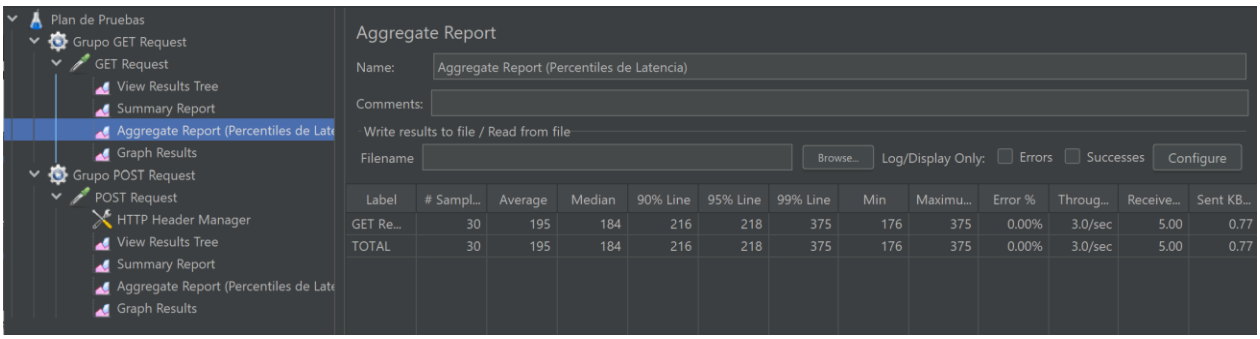
Baje Carga 3





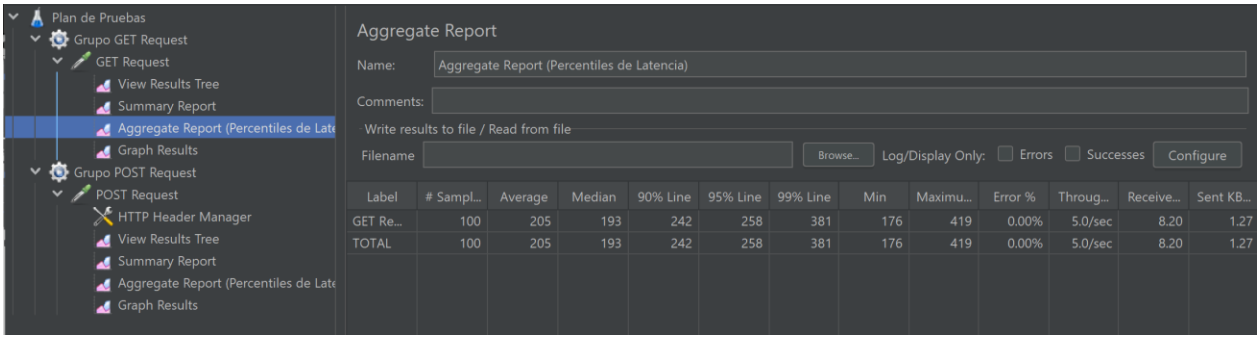
Baje Carga 4



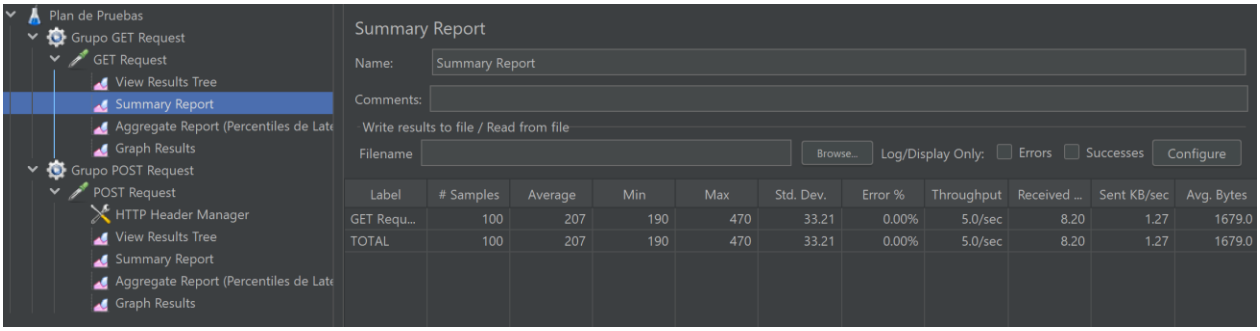


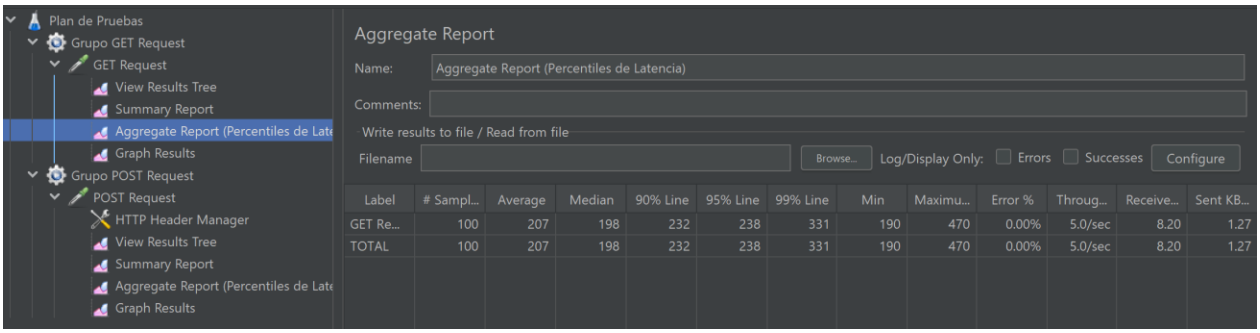
Carga Media 1



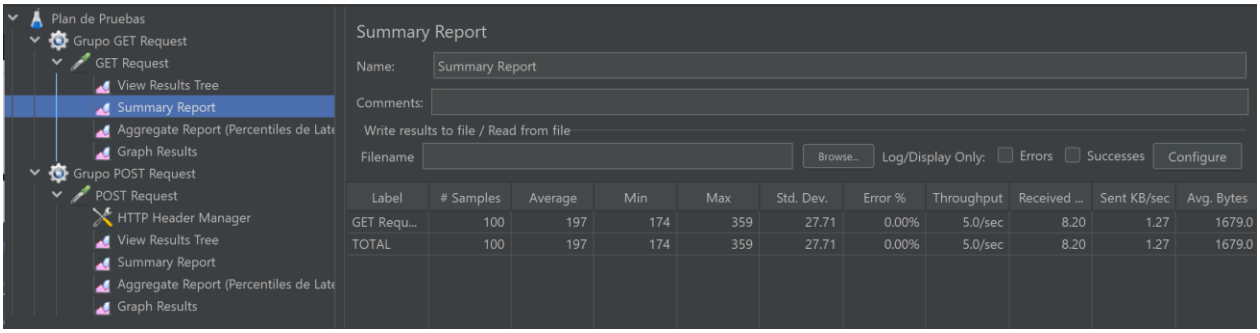


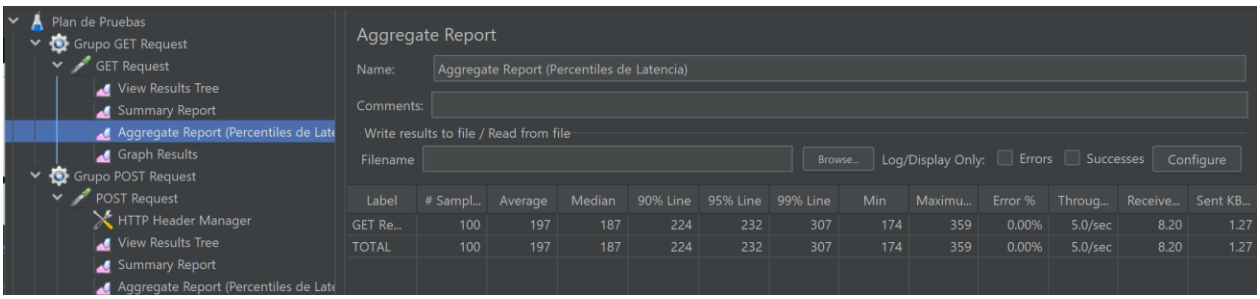
Carga Media 2



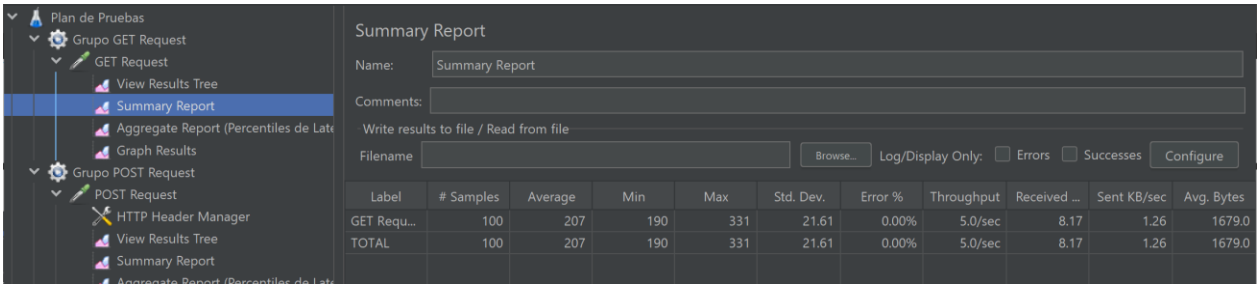


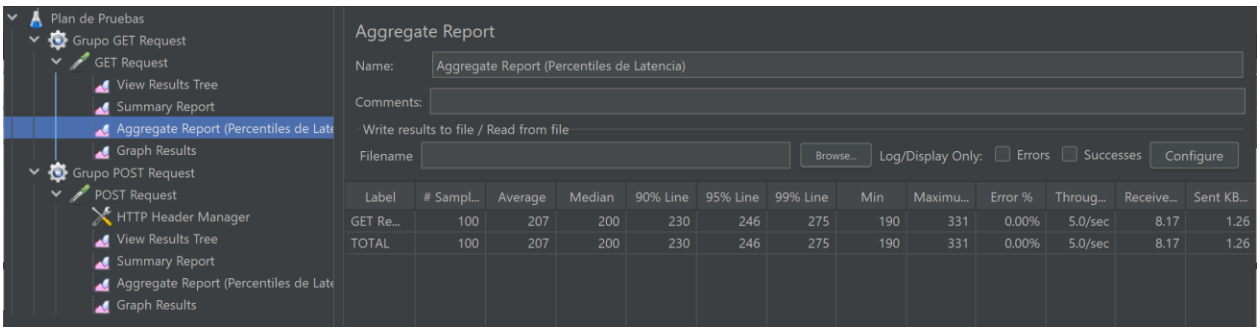
Carga Media 3



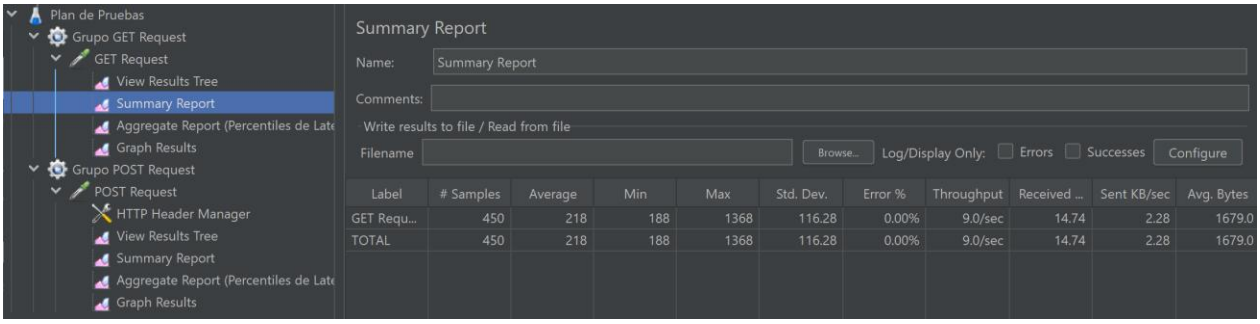


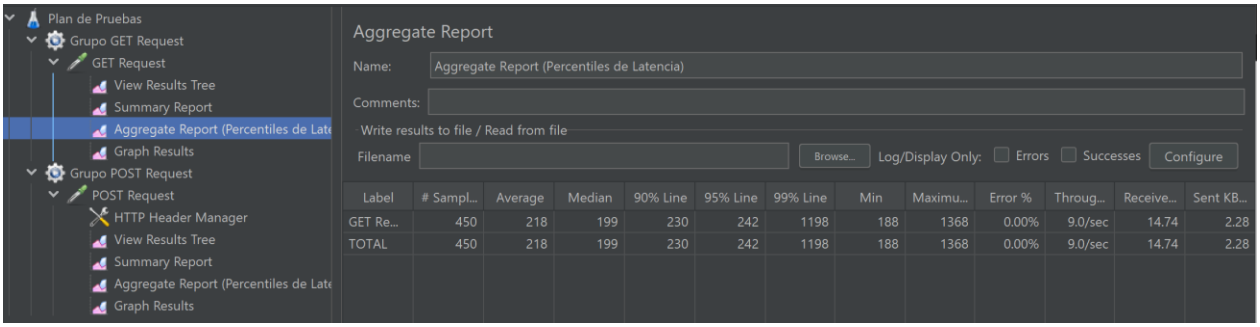
Carga Media 4



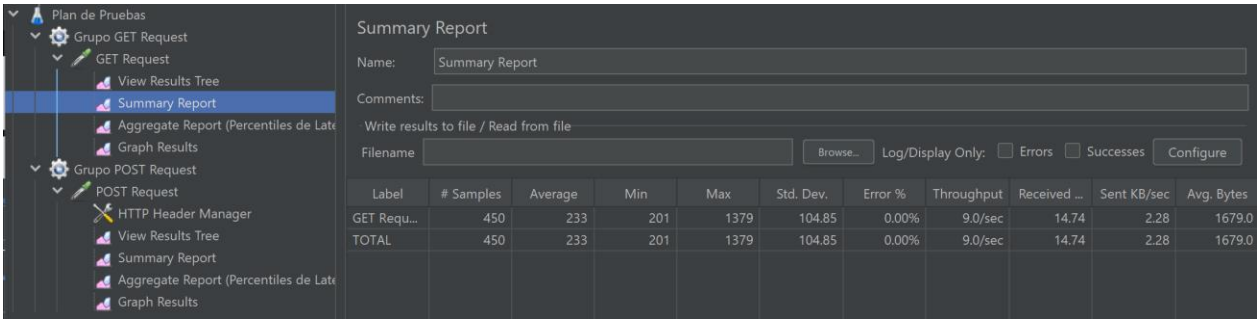


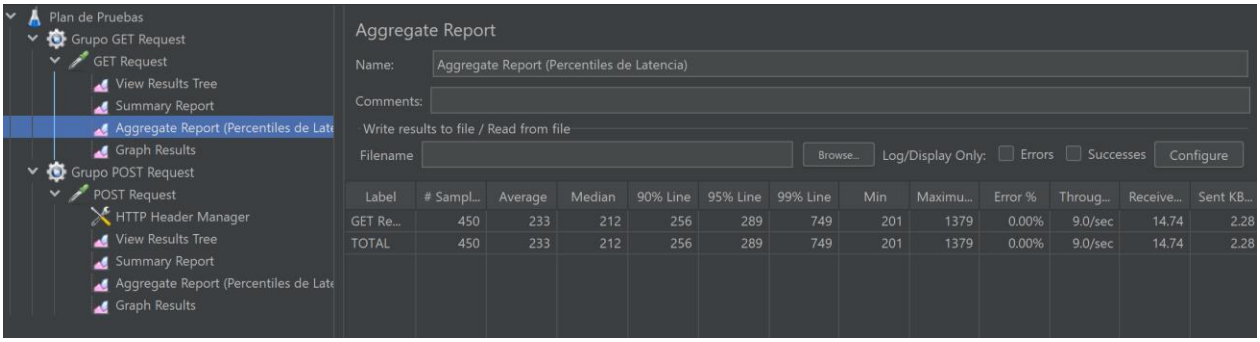
Operación Normal 1



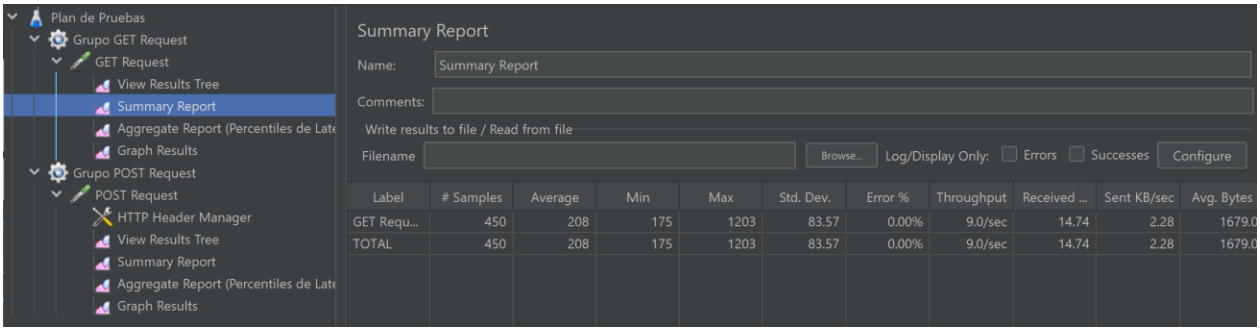


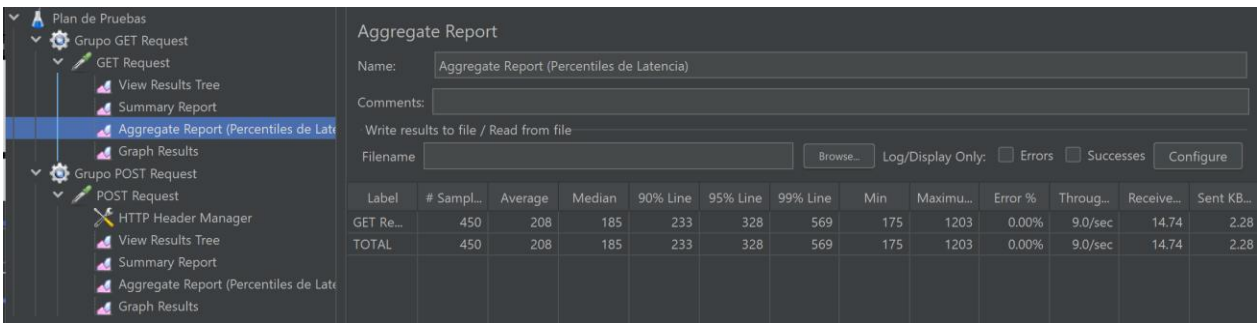
Operación Normal 2



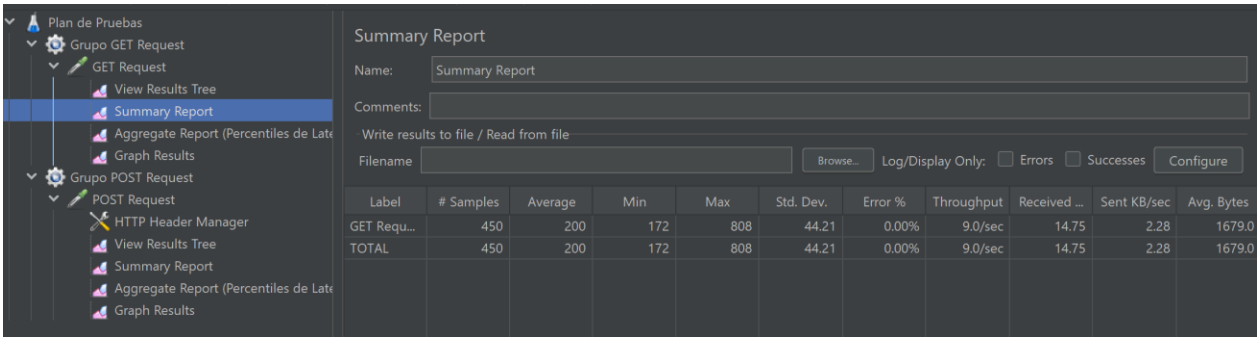


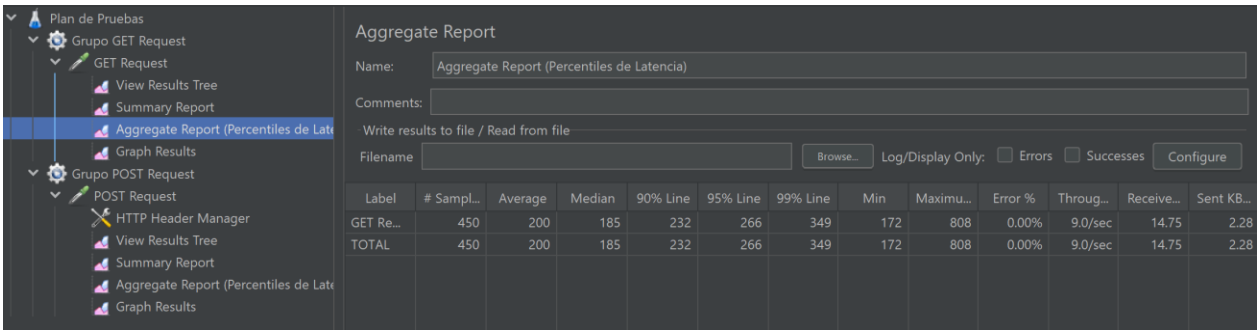
Operación Normal 3



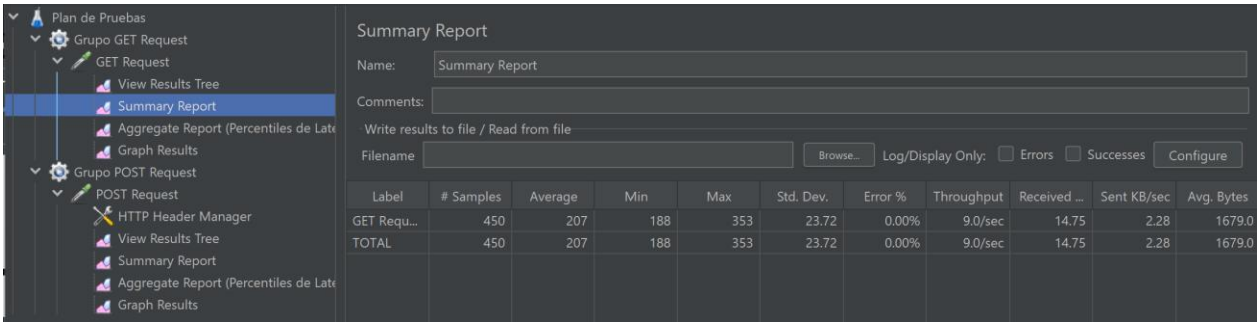


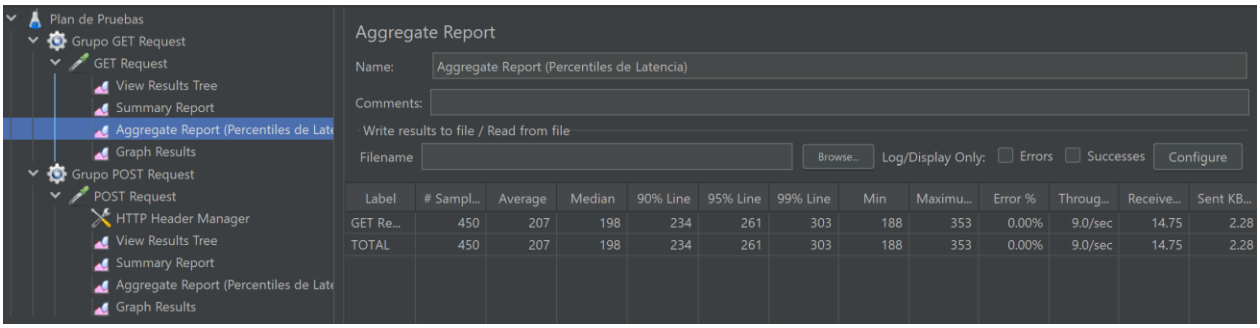
Operación Normal 4



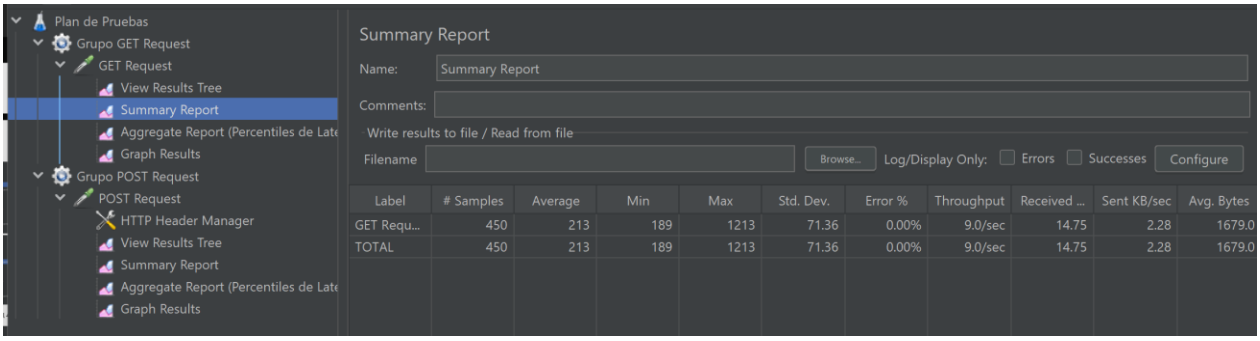


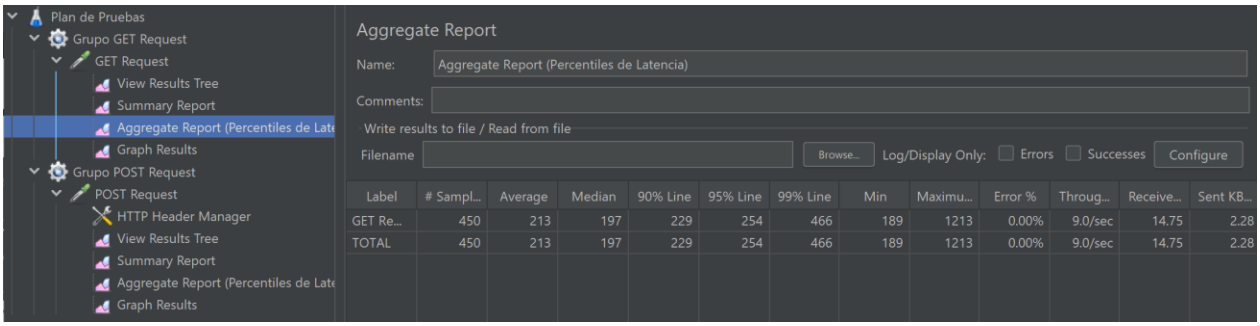
Operación Normal 5



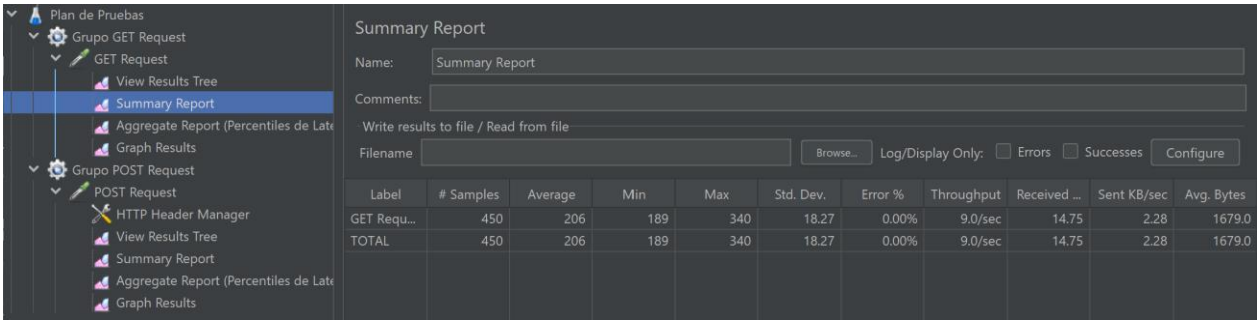


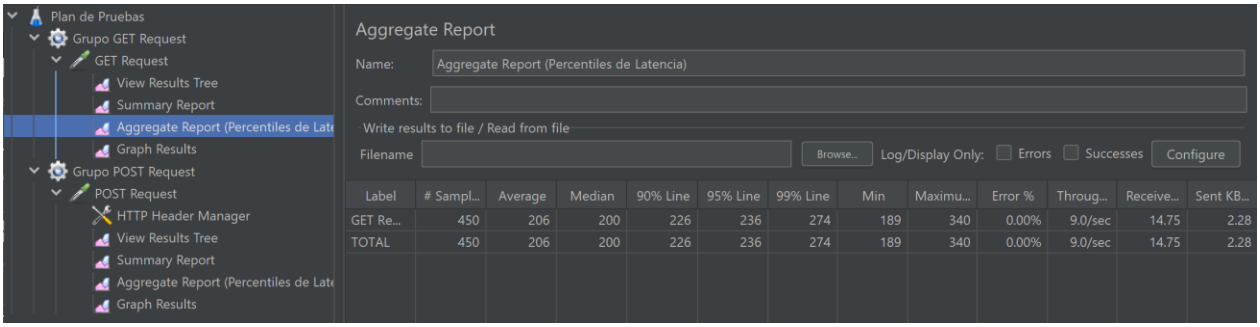
Operación Normal 6





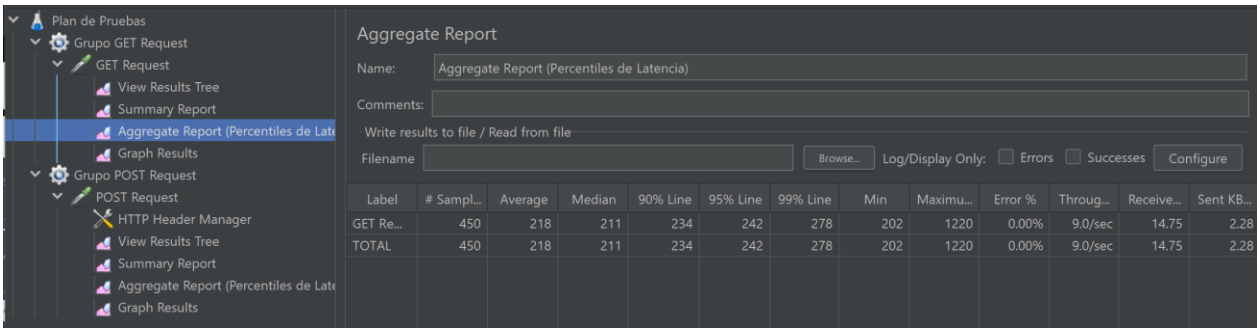
Operación Normal 7





Operación Normal 8





Alta Carga 1

```
ad_tests> python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      12821
Duracion real (s):   152.3
Throughput (req/s):  84.20
Latencia promedio:   9319.11 ms
Latencia p95:        32037.87 ms
Latencia p99:        50203.93 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Alta Carga 2

```
ad_tests> python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      12317
Duracion real (s):   150.0
Throughput (req/s):  82.14
Latencia promedio:   9298.71 ms
Latencia p95:        30972.02 ms
Latencia p99:        48438.79 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Alta Carga 3

```
ad_tests> python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      12193
Duracion real (s):   154.8
Throughput (req/s):  78.78
Latencia promedio:   9810.02 ms
Latencia p95:        33381.93 ms
Latencia p99:        52075.20 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Alta Carga 4

```
ad_tests> python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      11888
Duracion real (s):   150.7
Throughput (req/s):  78.89
Latencia promedio:   9626.04 ms
Latencia p95:        33320.42 ms
Latencia p99:        50510.61 ms
Error %:             0.01%
Resultados guardados en: results_get.csv
============================================================
```

Muy Alta Carga 1

```
ad_tests> python scripts/load_test.py --endpoint GET --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      13787
Duracion real (s):   221.0
Throughput (req/s):  62.38
Latencia promedio:   13480.23 ms
Latencia p95:        44488.67 ms
Latencia p99:        69042.22 ms
Error %:             0.01%
Resultados guardados en: results_get.csv
============================================================
```

Muy Alta Carga 2

```
ad_tests> python scripts/load_test.py --endpoint GET --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      14050
Duracion real (s):   219.3
Throughput (req/s):  64.07
Latencia promedio:   12738.88 ms
Latencia p95:        40766.84 ms
Latencia p99:        62248.26 ms
Error %:             0.01%
Resultados guardados en: results_get.csv
============================================================
```

Muy Alta Carga 3

```
ad_tests> python scripts/load_test.py --endpoint GET --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      14264
Duracion real (s):   216.0
Throughput (req/s):  66.04
Latencia promedio:   13731.57 ms
Latencia p95:        47199.45 ms
Latencia p99:        73149.50 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Muy Alta Carga 4

```
ad_tests> python scripts/load_test.py --endpoint GET --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      12706
Duracion real (s):   225.9
Throughput (req/s):  56.23
Latencia promedio:   14658.59 ms
Latencia p95:        47688.58 ms
Latencia p99:        71706.87 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés 1

```
ad_tests> python scripts/load_test.py --endpoint GET --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      19956
Duracion real (s):   422.7
Throughput (req/s):  47.21
Latencia promedio:   27804.17 ms
Latencia p95:        88353.03 ms
Latencia p99:        133900.76 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés 2

```
ad_tests> python scripts/load_test.py --endpoint GET --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      20725
Duracion real (s):   427.0
Throughput (req/s):  48.54
Latencia promedio:   28063.17 ms
Latencia p95:        84440.52 ms
Latencia p99:        129206.61 ms
Error %:             83.69%
Resultados guardados en: results_get.csv
============================================================
```

Estrés 3

```
ad_tests> python scripts/load_test.py --endpoint GET --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      25161
Duracion real (s):   411.8
Throughput (req/s):  61.10
Latencia promedio:   25615.82 ms
Latencia p95:        90216.31 ms
Latencia p99:        133689.87 ms
Error %:             100.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés 4

```
ad_tests> python scripts/load_test.py --endpoint GET --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      24047
Duracion real (s):   417.9
Throughput (req/s):  57.54
Latencia promedio:   24632.18 ms
Latencia p95:        85181.16 ms
Latencia p99:        130153.31 ms
Error %:             100.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 1

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      23706
Duracion real (s):   314.9
Throughput (req/s):  75.28
Latencia promedio:   35620.76 ms
Latencia p95:        117887.95 ms
Latencia p99:        172412.47 ms
Error %:             100.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 2

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      24373
Duracion real (s):   321.3
Throughput (req/s):  75.86
Latencia promedio:   37320.04 ms
Latencia p95:        123171.04 ms
Latencia p99:        180500.58 ms
Error %:             100.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 3

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      23206
Duracion real (s):   311.3
Throughput (req/s):  74.56
Latencia promedio:   33640.41 ms
Latencia p95:        113233.14 ms
Latencia p99:        162861.81 ms
Error %:             100.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 4

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Total requests:      21228
Duracion real (s):   317.4
Throughput (req/s):  66.87
Latencia promedio:   36008.39 ms
Latencia p95:        118009.90 ms
Latencia p99:        172599.01 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 5

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Total requests:      22122
Duracion real (s):   317.0
Throughput (req/s):  69.79
Latencia promedio:   36753.26 ms
Latencia p95:        119093.19 ms
Latencia p99:        174152.24 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 6

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      20001
Duracion real (s):   326.8
Throughput (req/s):  61.21
Latencia promedio:   38712.93 ms
Latencia p95:        126799.74 ms
Latencia p99:        183031.55 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 7

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      18238
Duracion real (s):   316.1
Throughput (req/s):  57.70
Latencia promedio:   35244.22 ms
Latencia p95:        112211.31 ms
Latencia p99:        163762.87 ms
Error %:             0.01%
Resultados guardados en: results_get.csv
============================================================
```

Estrés Fuerte 8

```
ad_tests> python scripts/load_test.py --endpoint GET --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — GET
============================================================
Total requests:      19157
Duracion real (s):   315.6
Throughput (req/s):  60.70
Latencia promedio:   35852.78 ms
Latencia p95:        118181.65 ms
Latencia p99:        174439.65 ms
Error %:             0.00%
Resultados guardados en: results_get.csv
============================================================
```

PRUEBAS POST

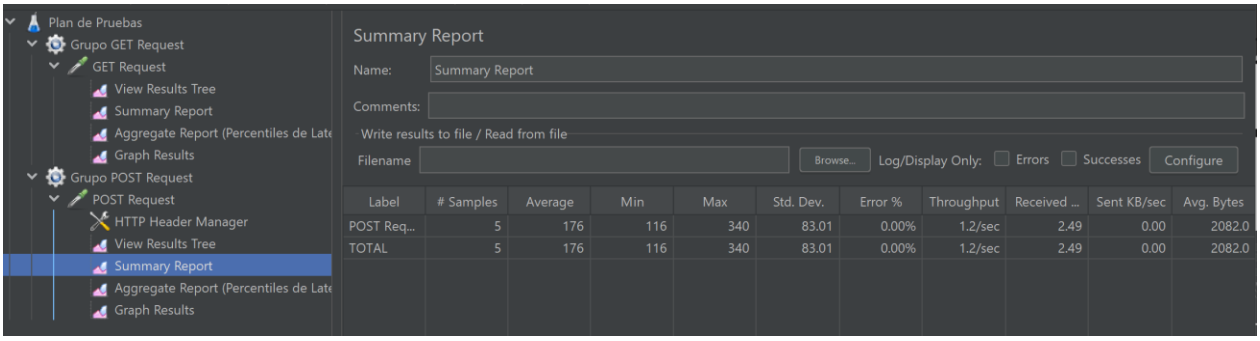
Escenario	Corrida	Threads / users	Ramp-up	Duración	p95 (ms)	p99 (ms)	Throughput	Error %
Smoke test	1	5	5s	—	340	340	1.2/sec	0.00
Smoke test	2	5	5s	—	324	324	1.2/sec	0.00
Smoke test	3	5	5s	—	246	246	1.2/sec	0.00
Smoke test	4	5	5s	—	346	346	1.2/sec	0.00
Baja carga	1	30	10s	—	133	307	3.1/sec	0.00
Baja carga	2	30	10s	—	176	316	3.1/sec	0.00
Baja carga	3	30	10s	—	213	589	3.1/sec	0.00
Baja carga	4	30	10s	—	150	375	3.1/sec	0.00
Carga media	1	100	20s	—	174	271	5.0/sec	0.00
Carga media	2	100	20s	—	157	252	5.0/sec	0.00
Carga media	3	100	20s	—	206	373	5.0/sec	0.00

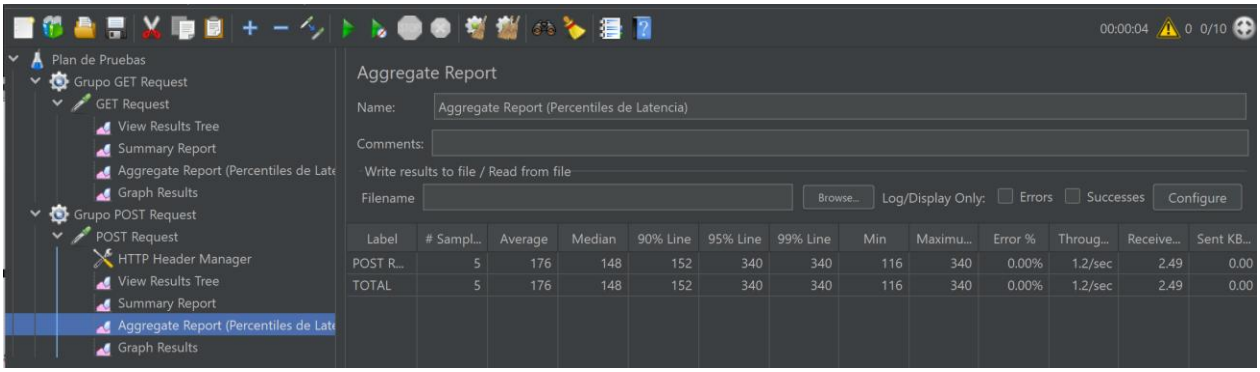
Carga media	4	100	20s	—	152	207	5.0/sec	0.00
Operación normal	1	450	50s	—	162	320	9.0/sec	0.00
Operación normal	2	450	50s	—	133	167	9.0/sec	0.00
Operación normal	3	450	50s	—	132	149	9.0/sec	0.00
Operación normal	4	450	50s	—	241	448	9.0/sec	0.00
Operación normal	5	450	50s	—	134	210	9.0/sec	0.00
Operación normal	6	450	50s	—	138	162	9.0/sec	0.00
Operación normal	7	450	50s	—	165	215	9.0/sec	0.00
Operación normal	8	450	50s	—	9664	11036	9.0/sec	0.00
Alta carga	1	1500	75s	60s	30627.16	48365.20	94.88/sec	100.00
Alta carga	2	1500	75s	60s	30391.32	45766.64	96.93/sec	100.00
Alta carga	3	1500	75s	60s	30967.25	46776.33	86.94/sec	100.00
Alta carga	4	1500	75s	60s	30524.79	46692.57	89.46/sec	100.00
Muy alta carga	1	3000	100s	60s	42607.72	65246.37	77.63/sec	100.00
Muy alta carga	2	3000	100s	60s	42078.67	64885.58	81.21/sec	100.00
Muy alta carga	3	3000	100s	60s	44829.02	65954.50	75.60/sec	100.00
Muy alta carga	4	3000	100s	60s	41534.85	62376.26	75.39/sec	100.00
Estrés	1	7500	150s	60s	87964.80	134476.22	59.91/sec	100.00

Estrés	2	7500	150s	60s	83471.89	126977.69	59.86/sec	100.00
Estrés	3	7500	150s	60s	86560.65	132383.88	59.61/sec	100.00
Estrés	4	7500	150s	60s	81561.76	122795.15	57.79/sec	100.00
Estrés fuerte (pico)	1	18000	200s	60s	116136.91	165464.66	51.55/sec	0.86
Estrés fuerte (pico)	2	18000	200s	60s	116433.00	170452.02	59.13/sec	0.00
Estrés fuerte (pico)	3	18000	200s	60s	121301.45	173298.17	60.62/sec	0.00
Estrés fuerte (pico)	4	18000	200s	60s	124627.35	180955.73	57.29/sec	0.00
Estrés fuerte (pico)	5	18000	200s	60s	122732.80	177252.80	58.32/sec	0.00
Estrés fuerte (pico)	6	18000	200s	60s	126036.14	181391.23	58.85/sec	0.00
Estrés fuerte (pico)	7	18000	200s	60s	122562.39	176266.31	61.57/sec	0.00
Estrés fuerte (pico)	8	18000	200s	60s	109419.97	162995.18	55.47/sec	0.00

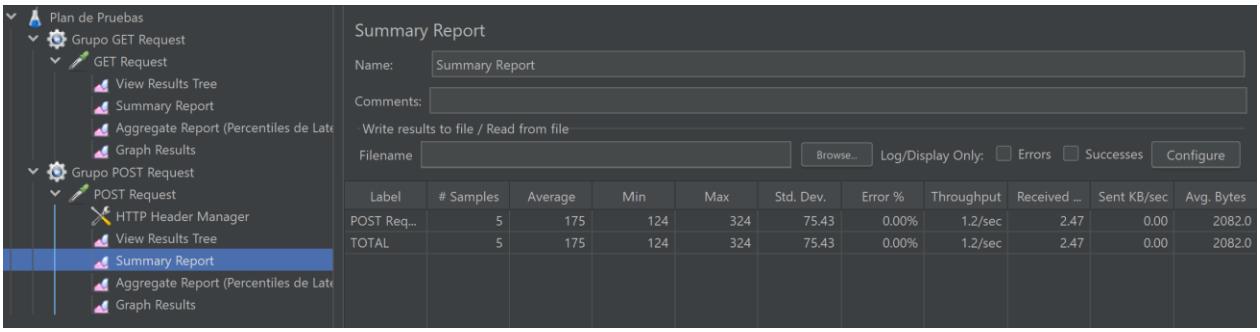
Evidencia

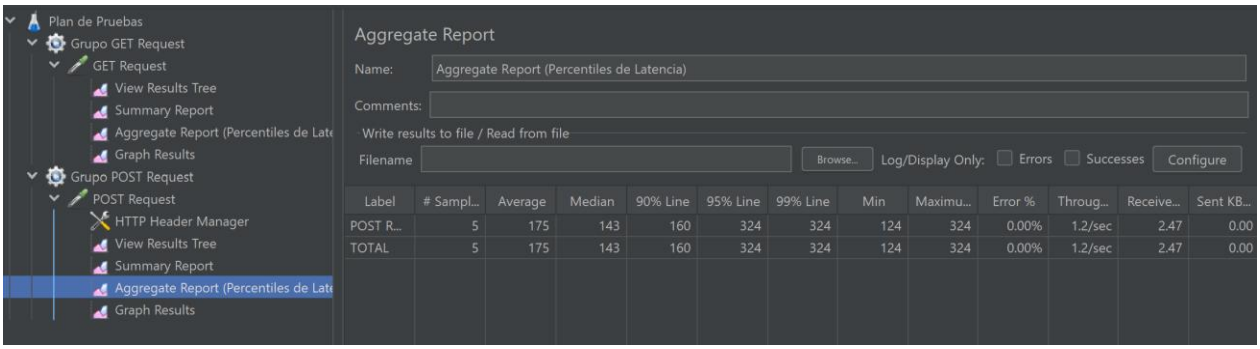
SmokeTest 1



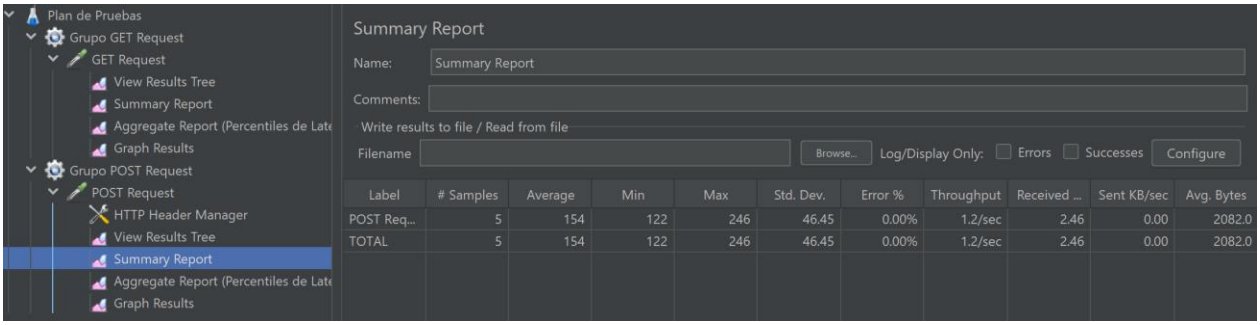


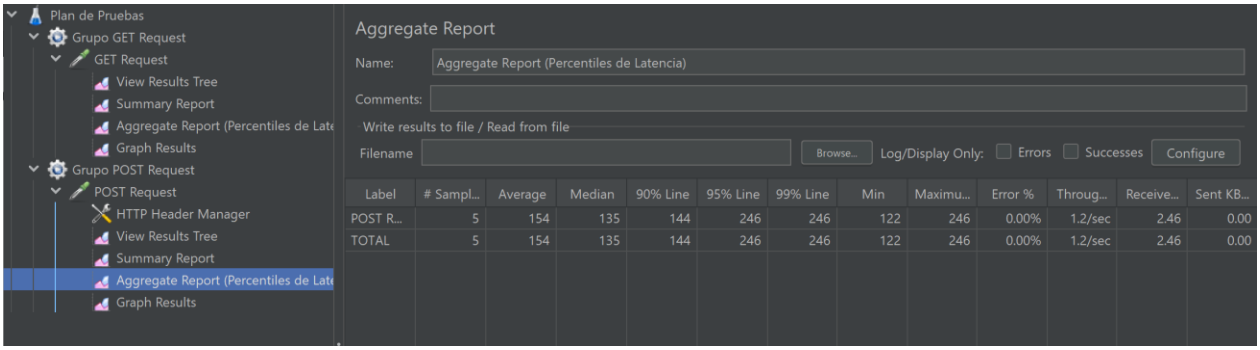
SmokeTest 2



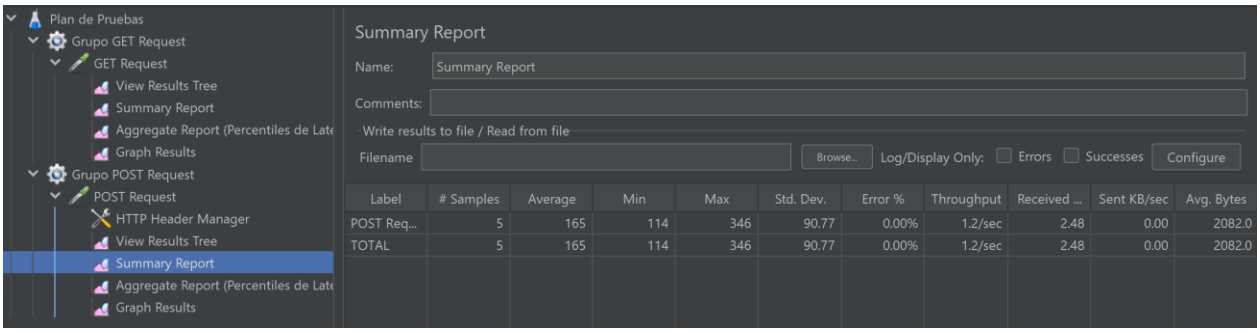


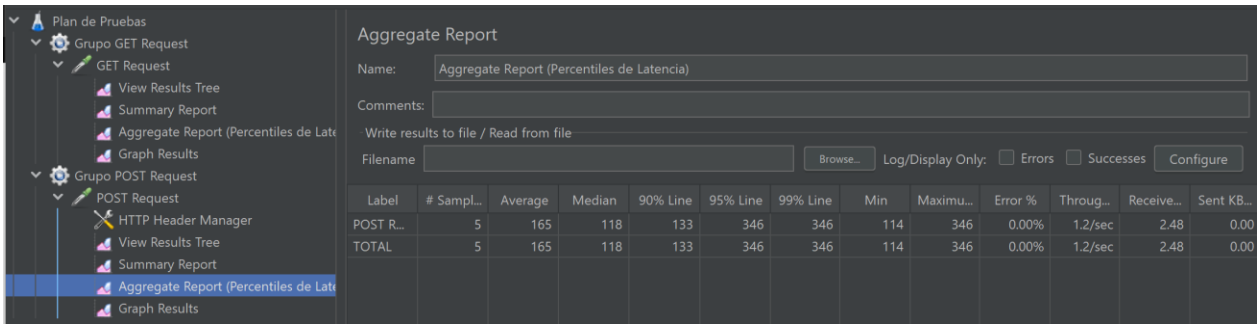
SmokeTest 3



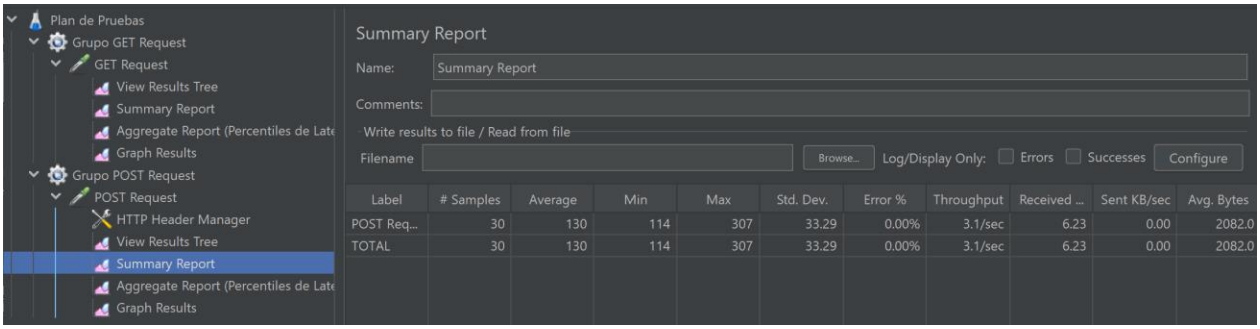


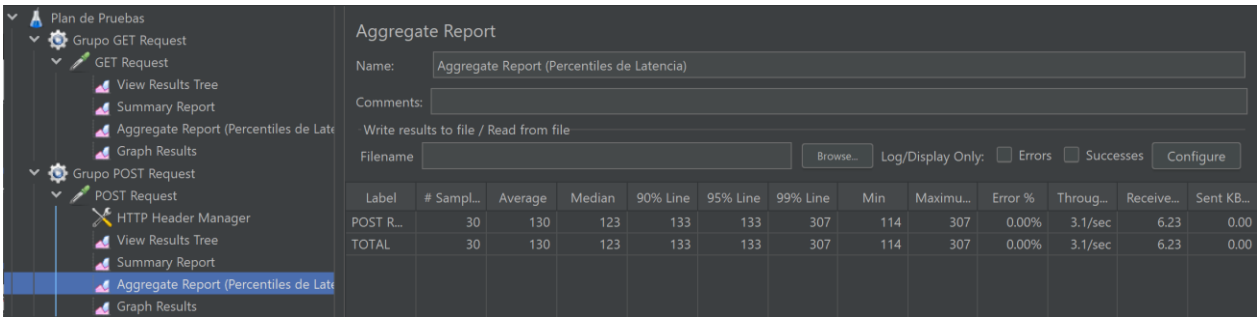
SmokeTest 4



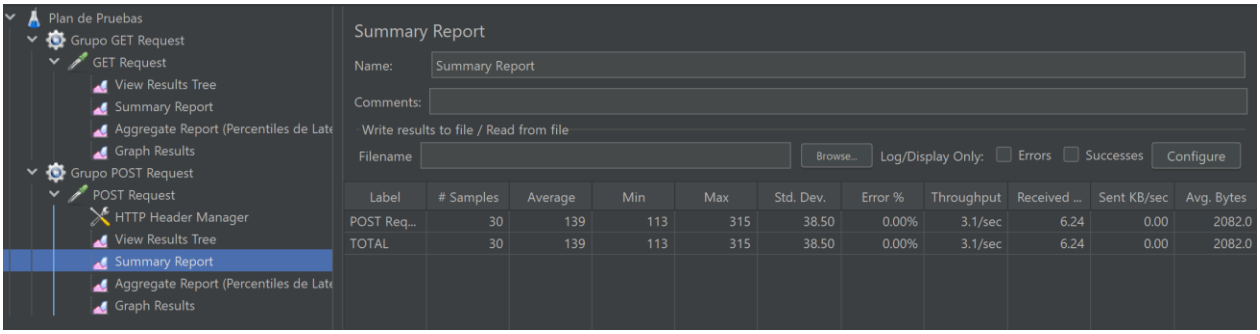


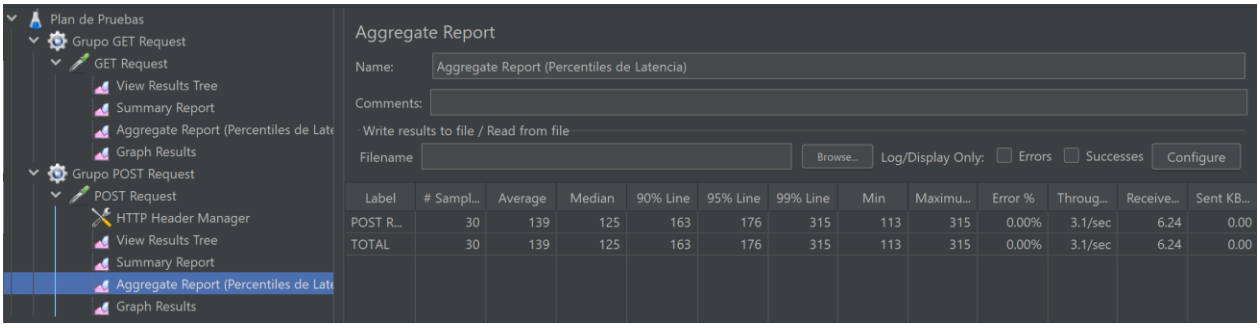
Baja Carga 1



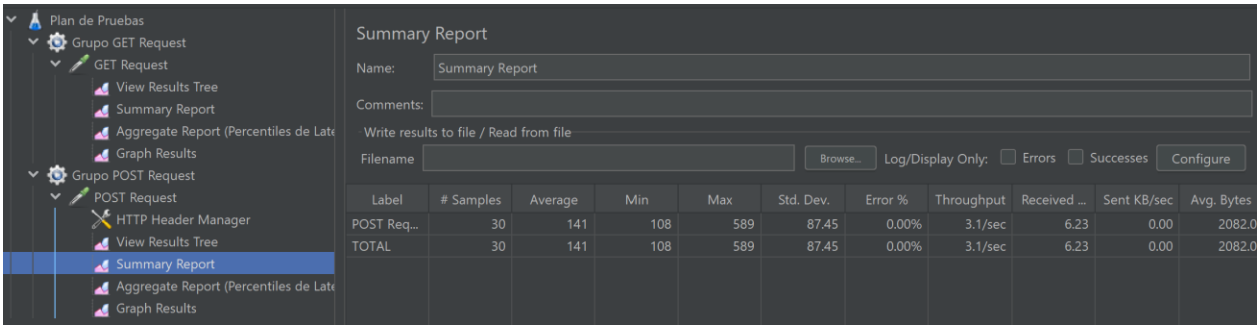


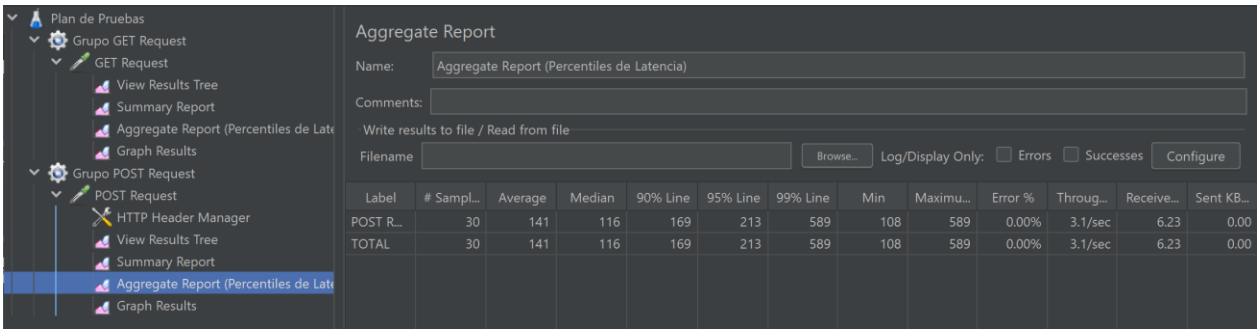
Baje Carga 2



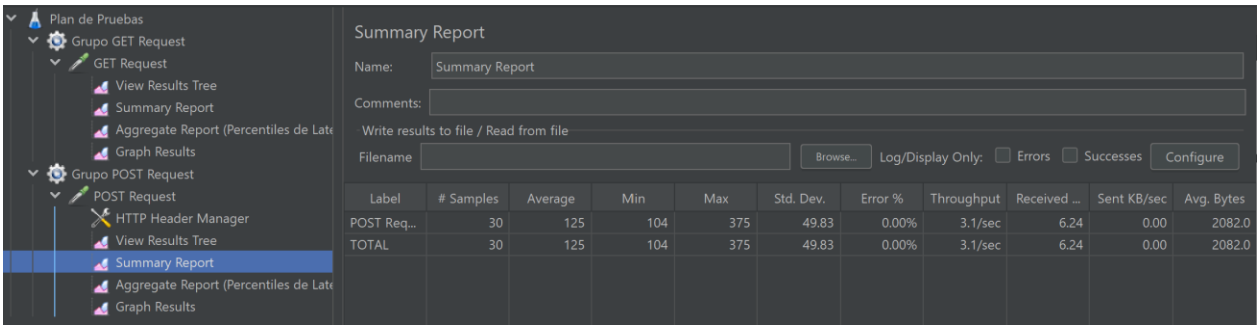


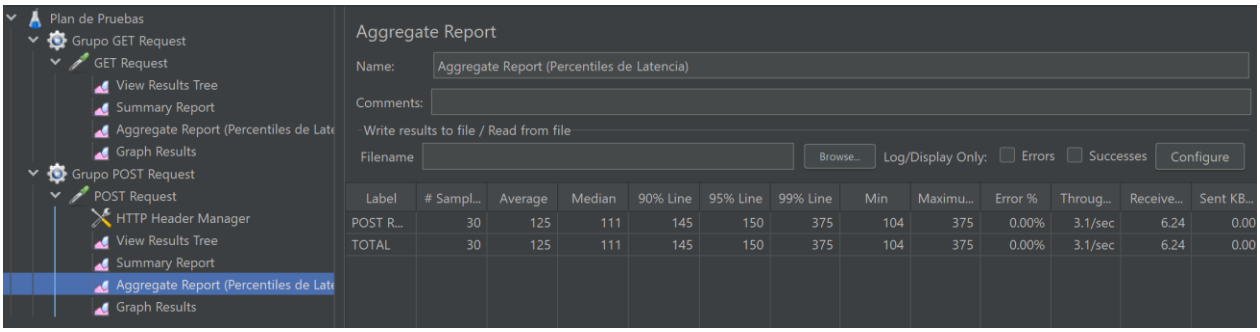
Baje Carga 3



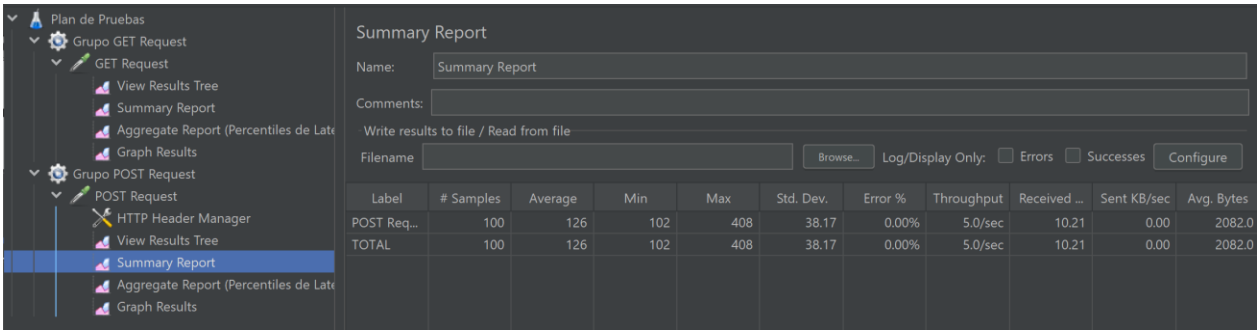


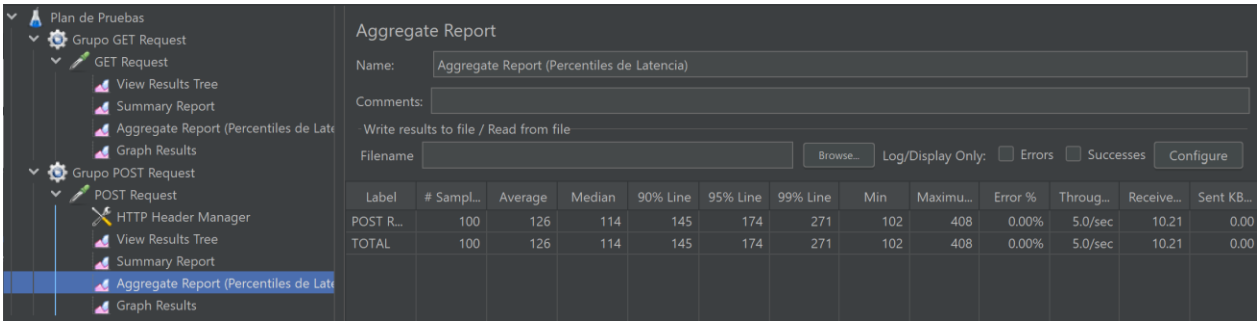
Baje Carga 4



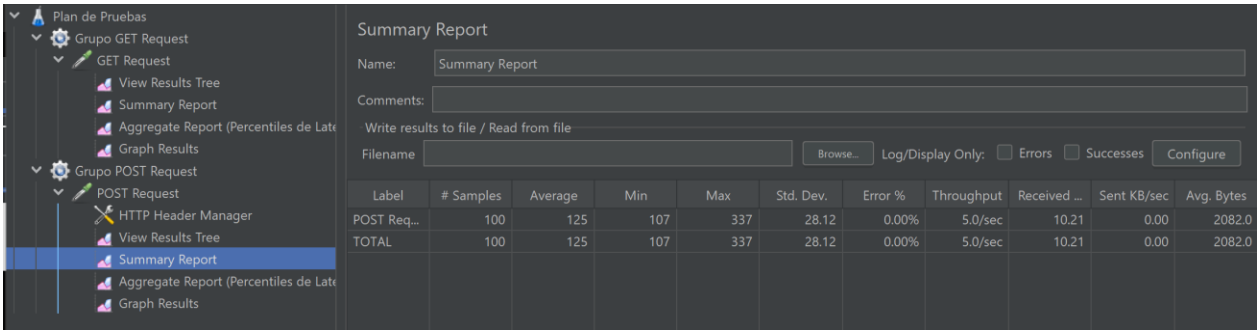


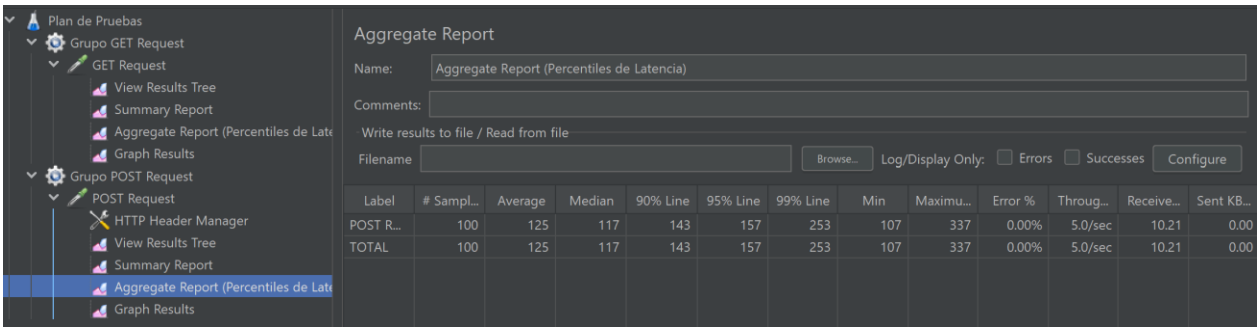
Carga Media 1



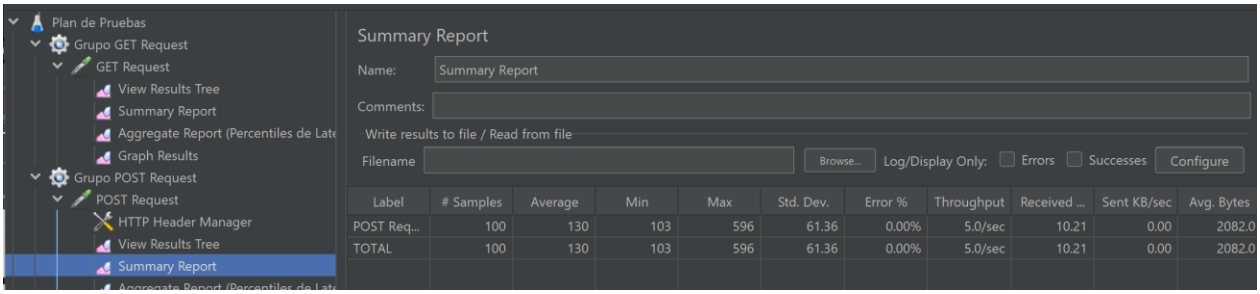


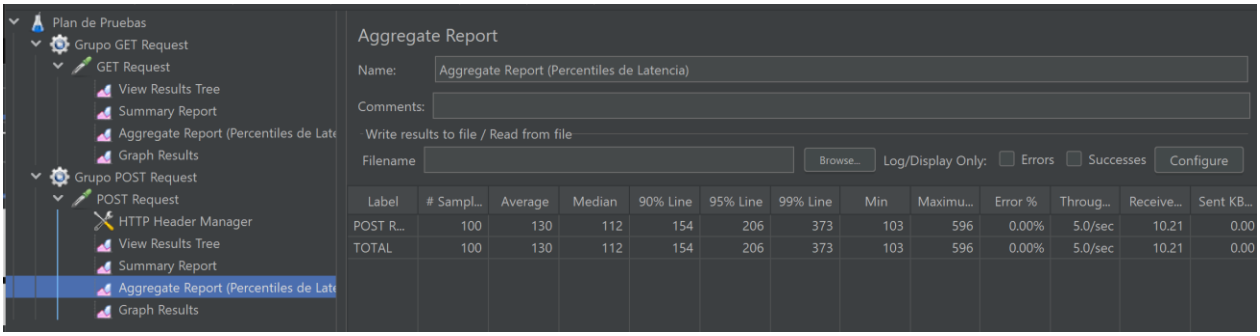
Carga Media 2



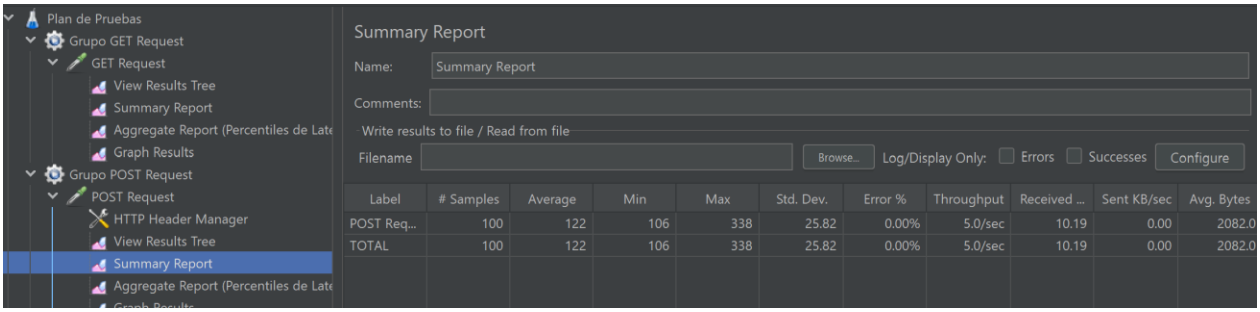


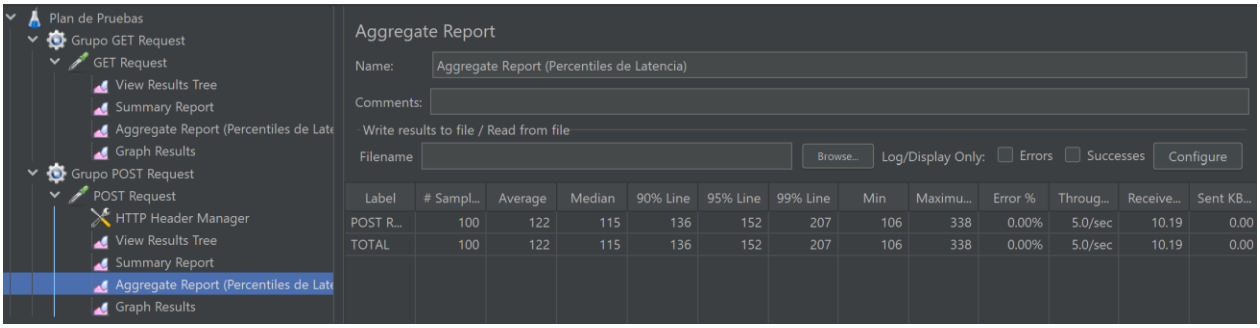
Carga Media 3



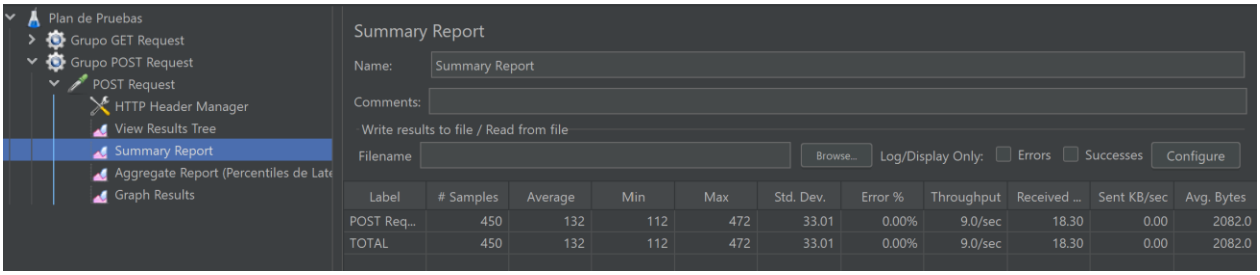


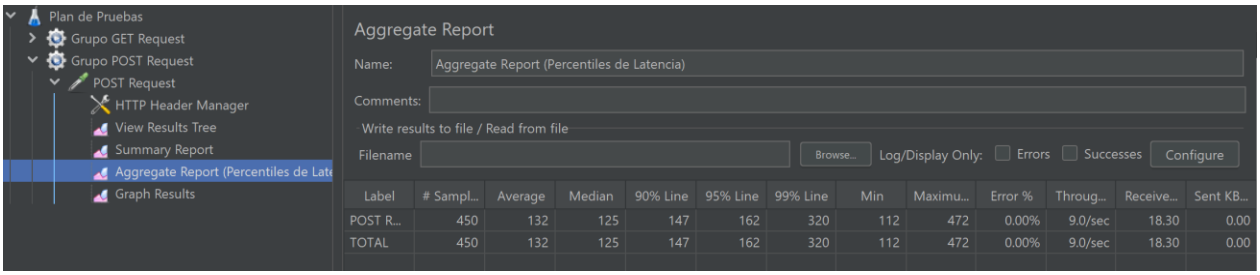
Carga Media 4



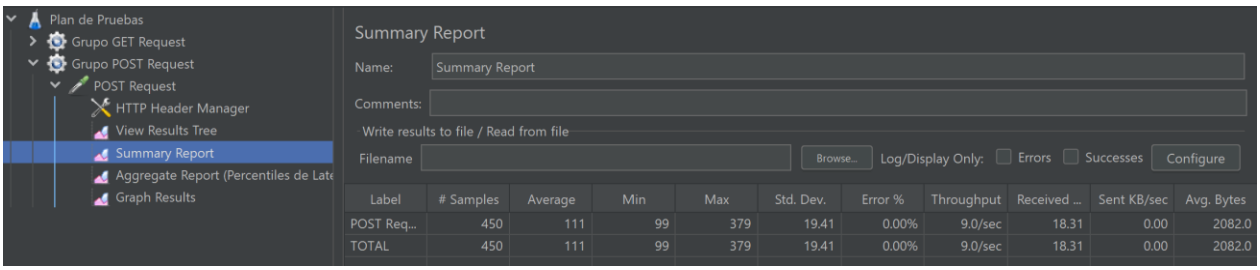


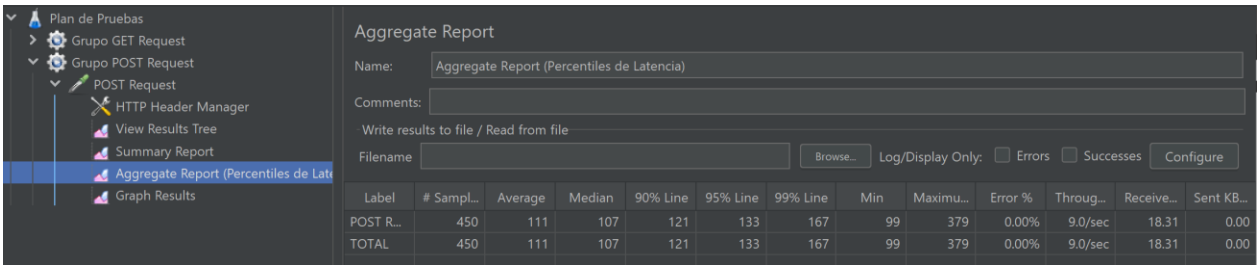
Operación Normal 1



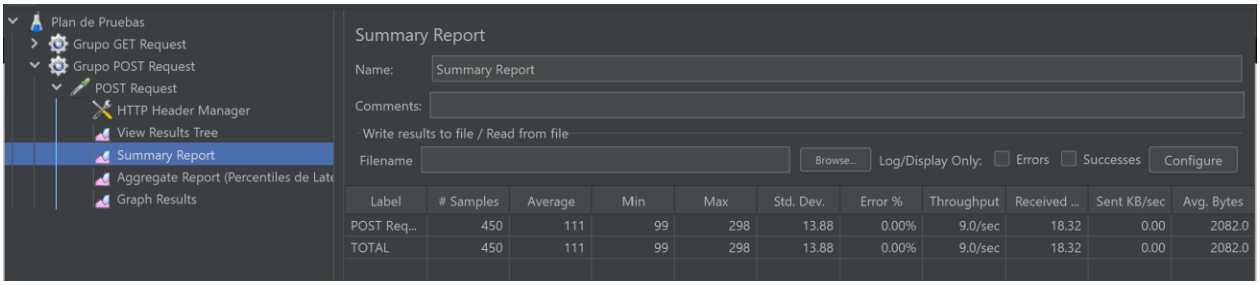


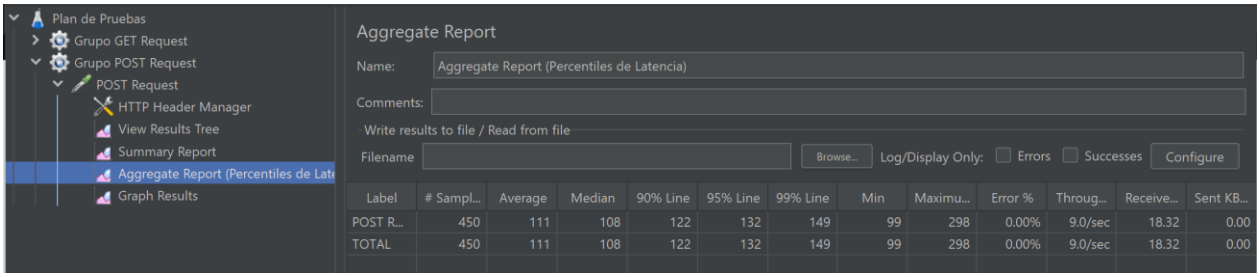
Operación Normal 2



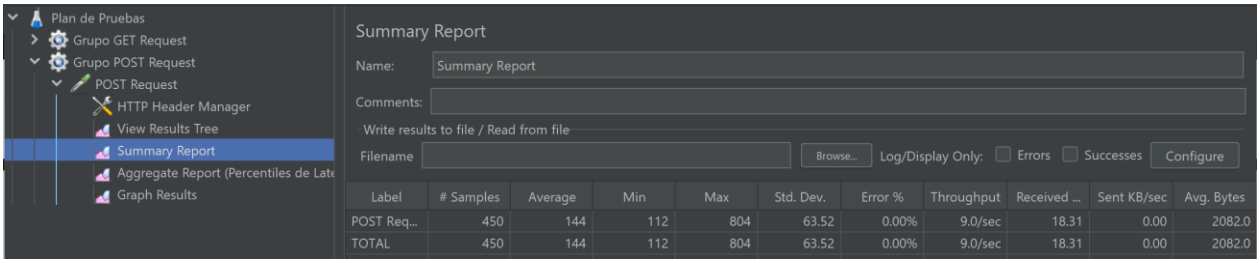


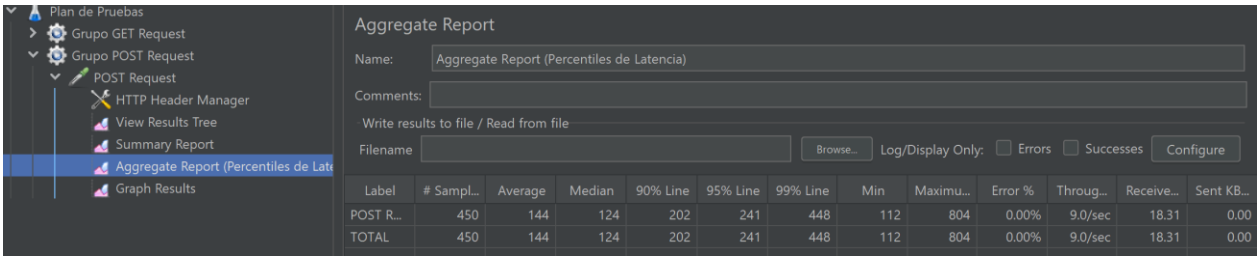
Operación Normal 3



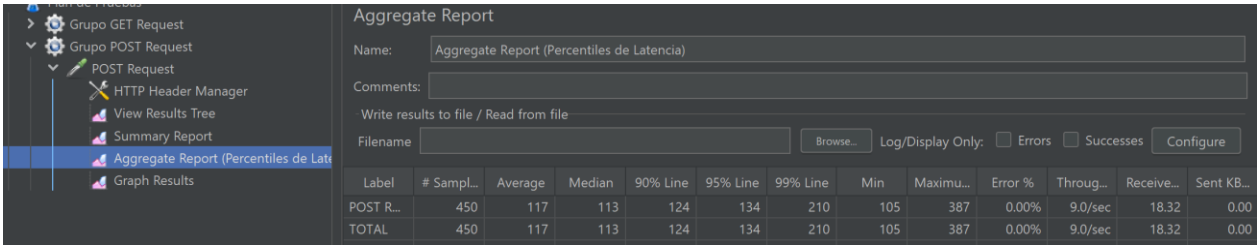


Operación Normal 4

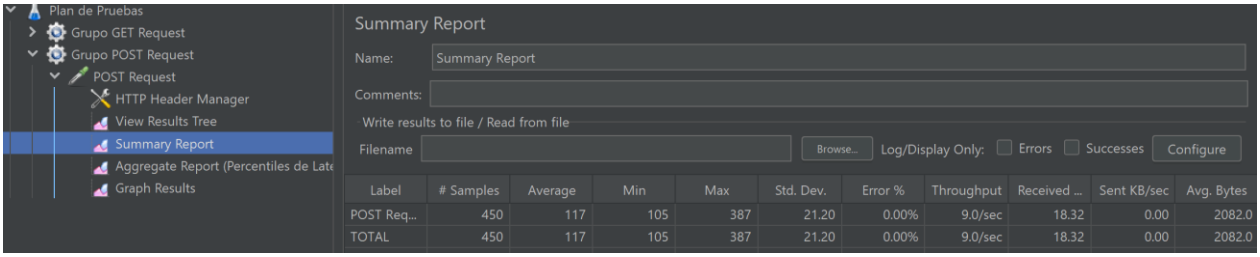




Operación Normal 5

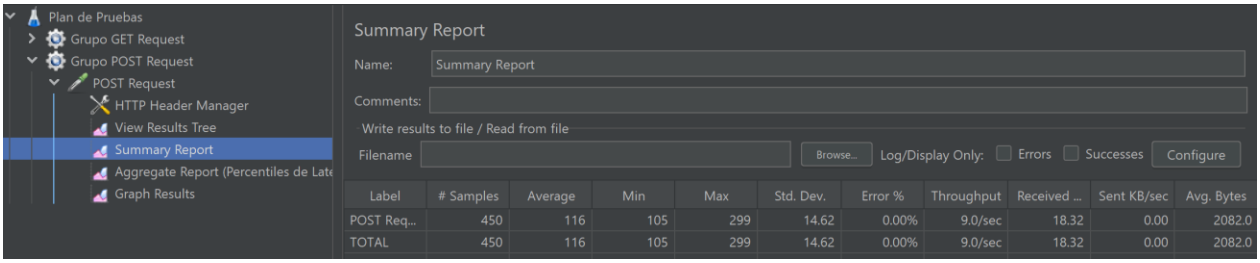


Label	# Sampl...	Average	Median	90% Line	95% Line	99% Line	Min	Maximu...	Error %	Throug...	Receive...	Sent KB...
POST R...	450	117	113	124	134	210	105	387	0.00%	9.0/sec	18.32	0.00
TOTAL	450	117	113	124	134	210	105	387	0.00%	9.0/sec	18.32	0.00

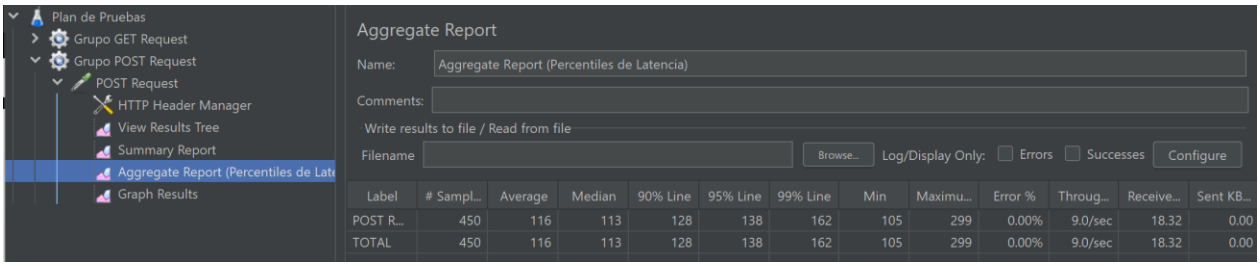


Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received ...	Sent KB/sec	Avg. Bytes
POST Req...	450	117	105	387	21.20	0.00%	9.0/sec	18.32	0.00	2082.0
TOTAL	450	117	105	387	21.20	0.00%	9.0/sec	18.32	0.00	2082.0

Operación Normal 6

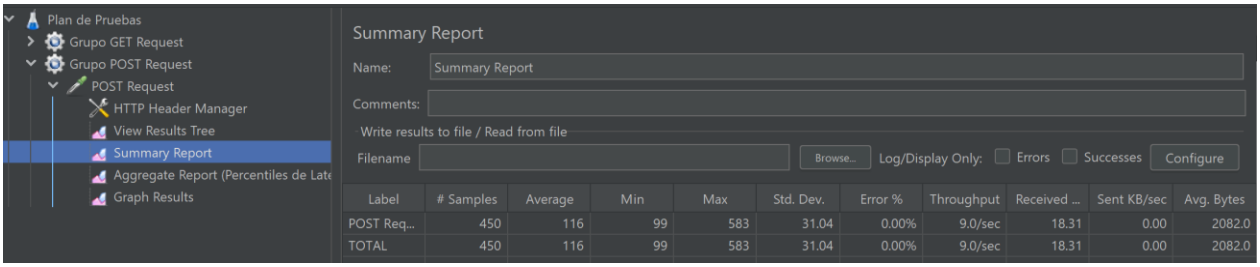


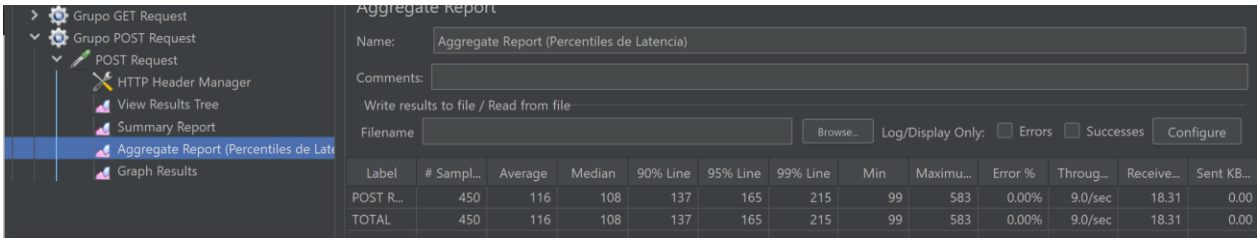
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received ...	Sent KB/sec	Avg. Bytes
POST Req...	450	116	105	299	14.62	0.00%	9.0/sec	18.32	0.00	2082.0
TOTAL	450	116	105	299	14.62	0.00%	9.0/sec	18.32	0.00	2082.0



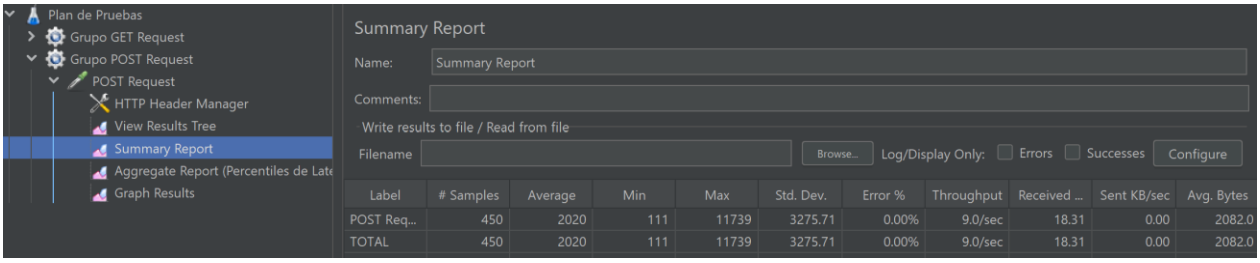
Label	# Sampl...	Average	Median	90% Line	95% Line	99% Line	Min	Maximu...	Error %	Throug...	Receive...	Sent KB...
POST R...	450	116	113	128	138	162	105	299	0.00%	9.0/sec	18.32	0.00
TOTAL	450	116	113	128	138	162	105	299	0.00%	9.0/sec	18.32	0.00

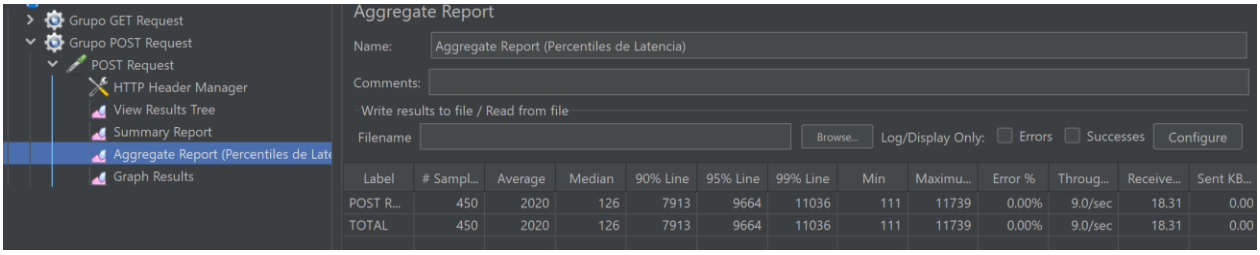
Operación Normal 7





Operación Normal 8





Alta Carga 1

```
ad_tests> python scripts/load_test.py --endpoint POST --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      14097
Duracion real (s):   148.6
Throughput (req/s):  94.88
Latencia promedio:   8808.43 ms
Latencia p95:        30627.16 ms
Latencia p99:        48365.20 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Alta Carga 2

```
ad_tests> python scripts/load_test.py --endpoint POST --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      14464
Duracion real (s):   149.2
Throughput (req/s):  96.93
Latencia promedio:   8780.88 ms
Latencia p95:        30391.32 ms
Latencia p99:        45766.64 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Alta Carga 3

```
ad_tests> python scripts/load_test.py --endpoint POST --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      13023
Duracion real (s):   149.8
Throughput (req/s):  86.94
Latencia promedio:   9109.25 ms
Latencia p95:        30967.25 ms
Latencia p99:        46776.33 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Alta Carga 4

```
ad_tests> python scripts/load_test.py --endpoint POST --users 1500 --ramp-up 75 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      13360
Duracion real (s):   149.3
Throughput (req/s):  89.46
Latencia promedio:   8900.44 ms
Latencia p95:        30524.79 ms
Latencia p99:        46692.57 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Muy Alta Carga 1

```
ad_tests> python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      16202
Duracion real (s):   208.7
Throughput (req/s):  77.63
Latencia promedio:   12699.18 ms
Latencia p95:        42607.72 ms
Latencia p99:        65246.37 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
```

Muy Alta Carga 2

```
ad_tests> python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      16694
Duracion real (s):   205.6
Throughput (req/s):  81.21
Latencia promedio:   12567.17 ms
Latencia p95:        42078.67 ms
Latencia p99:        64885.58 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Muy Alta Carga 3

```
ad_tests> python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      16048
Duracion real (s):   212.3
Throughput (req/s):  75.60
Latencia promedio:   13130.76 ms
Latencia p95:        44829.02 ms
Latencia p99:        65954.50 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Muy Alta Carga 4

```
ad_tests> python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      15950
Duracion real (s):   211.6
Throughput (req/s):  75.39
Latencia promedio:   12433.90 ms
Latencia p95:        41534.85 ms
Latencia p99:        62376.26 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés 1

```
ad_tests> python scripts/load_test.py --endpoint POST --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      24561
Duracion real (s):   409.9
Throughput (req/s):  59.91
Latencia promedio:   25459.98 ms
Latencia p95:        87964.80 ms
Latencia p99:        134476.22 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés 2

```
ad_tests> python scripts/load_test.py --endpoint POST --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      24640
Duracion real (s):   411.6
Throughput (req/s):  59.86
Latencia promedio:   24413.40 ms
Latencia p95:        83471.89 ms
Latencia p99:        126977.69 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés 3

```
ad_tests> python scripts/load_test.py --endpoint POST --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      24407
Duracion real (s):   409.4
Throughput (req/s):  59.61
Latencia promedio:   24858.36 ms
Latencia p95:        86560.65 ms
Latencia p99:        132383.88 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés 4

```
ad_tests> python scripts/load_test.py --endpoint POST --users 7500 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      23892
Duracion real (s):   413.4
Throughput (req/s):  57.79
Latencia promedio:   23826.96 ms
Latencia p95:        81561.76 ms
Latencia p99:        122795.15 ms
Error %:             100.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 1

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      16481
Duracion real (s):   319.7
Throughput (req/s):  51.55
Latencia promedio:   40249.39 ms
Latencia p95:        116136.91 ms
Latencia p99:        165464.66 ms
Error %:             0.86%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 2

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      18826
Duracion real (s):   318.4
Throughput (req/s):  59.13
Latencia promedio:   36143.23 ms
Latencia p95:        116433.00 ms
Latencia p99:        170452.02 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 3

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      19211
Duracion real (s):   316.9
Throughput (req/s):  60.62
Latencia promedio:   37777.64 ms
Latencia p95:        121301.45 ms
Latencia p99:        173298.17 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 4

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      19026
Duracion real (s):   332.1
Throughput (req/s):  57.29
Latencia promedio:   38399.26 ms
Latencia p95:        124627.35 ms
Latencia p99:        180955.73 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 5

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      18722
Duracion real (s):   321.0
Throughput (req/s):  58.32
Latencia promedio:   37852.76 ms
Latencia p95:        122732.80 ms
Latencia p99:        177252.80 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 6

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      18614
Duracion real (s):   316.3
Throughput (req/s):  58.85
Latencia promedio:   37862.87 ms
Latencia p95:        126036.14 ms
Latencia p99:        181391.23 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 7

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      19566
Duracion real (s):   317.8
Throughput (req/s):  61.57
Latencia promedio:   37976.15 ms
Latencia p95:        122562.39 ms
Latencia p99:        176266.31 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
============================================================
```

Estrés Fuerte 8

```
ad_tests> python scripts/load_test.py --endpoint POST --users 18000 --ramp-up 200 --duration 60
============================================================
Resumen de la prueba — POST
============================================================
Total requests:      17301
Duracion real (s):   311.9
Throughput (req/s):  55.47
Latencia promedio:   35250.22 ms
Latencia p95:        109419.97 ms
Latencia p99:        162995.18 ms
Error %:             0.00%
Resultados guardados en: results_post.csv
============================================================
```

Análisis

1. **¿Cuál fue el punto de inflexión y cuál ASR se rompió primero?**

El primero en romperse es ASR2 (error ≤2%) en el POST, ya desde Alta carga (1500 threads) con 100% de error, mucho antes que cualquier problema de latencia. En GET, el sistema aguanta razonablemente hasta Muy Alta Carga (3000) y colapsa en Estrés (7500 threads).

2. **Con base en los resultados, ¿el diseño monolítico favorece el cumplimiento de los ASRs? Explique.**

No. Los datos lo muestran claramente: el sistema pasa de funcionar perfecto (0% error, latencias de cientos de ms) a colapsar totalmente (100% error, decenas de segundos de latencia) en un salto muy pequeño de concurrencia. Un monolito con una sola base de datos compartida no tiene forma de aislar o amortiguar ese colapso — todas las instancias de app terminan compitiendo por el mismo pool de conexiones a la misma DB.

3. **¿Qué cambios de arquitectura (estilos/tácticas/patrones) propondría para cumplir los ASRs?**

- Pool de conexiones dedicado (PgBouncer) para no agotar conexiones directas a Postgres.
- Cache (Redis) para la consulta GET de productos en promoción, que es de lectura repetitiva.
- Cola asíncrona (SQS/RabbitMQ) para el POST de pedidos: aceptar el pedido rápido y procesarlo en background, devolviendo éxito inmediato en vez de esperar toda la transacción.
- Separar el monolito en servicios más pequeños (catálogo/lectura vs pedidos/escritura) para que un pico de escritura no tumbe las lecturas.
- Rate limiting / circuit breaker para degradar controladamente en vez de colapsar a 100% de error.

4. **¿El patrón de degradación fue gradual o abrupto? ¿Cuál fue el cuello de botella más probable?**

Abrupto. El sistema se mantiene estable (0% error) hasta cierto umbral y luego salta directamente a 83-100% de error, no hay una zona intermedia de degradación progresiva. Esto es típico de un cuello de botella por agotamiento de un recurso finito (probablemente el pool de conexiones de PostgreSQL o el límite de conexiones máximas del contenedor Docker), que funciona bien hasta que se llena, y ahí todo empieza a fallar de golpe.

5. **¿Qué endpoint degradó primero y por qué ocurrió?**

El **POST** degrada primero (ya en 1500 threads). Cada POST hace una transacción con múltiples inserciones (pedido + ítems), lo que mantiene la conexión a la base de datos ocupada por más tiempo y compite por bloqueos (locks) de escritura. El GET es una consulta de solo lectura, más

liviana y sin bloqueos, por lo que aguanta más concurrencia antes de saturar el mismo recurso compartido.

6. **Existen múltiples algoritmos que se pueden usar para el balanceo de cargas, cada uno responde a características del tráfico que pueda tener el servicio a balancear, número de usuarios y comportamiento de estos con los sistemas o incluso características de hardware. Investigue qué algoritmo usa ALB y haga una tabla comparativa en múltiples aspectos con los algoritmos Round-robbin, Hashing por IP, Least conn, Least response. En esta tabla debe comparar las características de los algoritmos aplicados a Cheapest**

AWS ALB usa por defecto Round Robin (o "Least Outstanding Requests" si se habilita explícitamente esa política).

Algoritmo	Cómo decide	Ventaja para Cheapest	Desventaja para Cheapest
Round Robin (default de ALB)	Reparte en orden secuencial, uno a cada instancia por turno	Simple, reparte parejo cuando las requests son similares	No tiene en cuenta si una instancia ya está saturada por una consulta pesada
Hashing por IP	Misma IP siempre va a la misma instancia	Útil si se necesitara "sticky sessions"	Para Cheapest no aporta nada, y puede sobrecargar una instancia si muchos tenderos de la misma IP/zona pegan al mismo tiempo
Least Connections	Envía a la instancia con menos conexiones activas	Mejor para Cheapest en el pico de promociones, ya que evita mandar más carga a una instancia ya ocupada con pedidos pesados	Requiere que el balanceador monitoree conexiones activas constantemente (más overhead)
Least Response Time	Envía a la instancia que responde más rápido últimamente	El mejor teóricamente para mezclar GET livianos y POST pesados, evita instancias lentas	Más complejo de implementar y AWS ALB no lo soporta nativamente sin configuración avanzada (Least Outstanding Requests es lo más cercano)

Para Cheapest, dado que el POST es mucho más pesado que el GET, Least Connections o la opción de ALB "Least Outstanding Requests" sería más adecuado que el Round Robin por defecto, porque evitaría enviar más pedidos pesados a una instancia que ya está ocupada procesando otros.

7. **Compare cuantitativamente estos resultados con los del Lab 2 (monolito local): ¿cómo cambiaron el punto de inflexión, el p99, el throughput y el error % al separar App y DB**

y replicar la App detrás del ALB? ¿La mejora (o la falta de ella) es coherente con el cuello de botella que identificó?

La migración a AWS NO mejoró el punto de inflexión, de hecho, lo empeoró para el POST. Esto es completamente coherente con el cuello de botella que identificamos: si el problema real es el pool de conexiones a PostgreSQL (10 conexiones por defecto) y las transacciones de escritura del POST, entonces separar la app en 3 instancias detrás del ALB multiplicó el problema en vez de resolverlo, ahora son 3 procesos Node.js independientes, cada uno abriendo su propio pool de hasta 10 conexiones hacia la *misma* instancia de base de datos, compitiendo entre sí por el mismo recurso escaso, más la latencia de red adicional entre App y DB que antes no existía (todo era local). El ALB solo reparte las peticiones de entrada; no resuelve que las tres instancias sigan ahogando a una única base de datos con capacidad limitada, validando exactamente lo que dijiste en la Pregunta 2 de este Lab 3: el ALB mejora disponibilidad, pero sobreestima la escalabilidad real cuando el cuello de botella sigue siendo la base de datos compartida.

Prompt utilizado para generar el script en laboratorio 2

```
Vamos a realizar pruebas de performance. Necesito un script en Python
para ejecutar
pruebas de carga contra un backend NestJS usando HTTP.
Contexto:
- Base URL: http://localhost:3000
- Endpoints a probar:
  - GET (lectura pesada, múltiples JOINs vía EXISTS):
    /logistics/tenderos/productos-
disponibles?tiendaId=<uuid>&zona=<zona>
  - POST (escritura transaccional): /logistics/pedidos
- IDs fijos disponibles en el seed de la base
(src/datasources/seed.sql):
  tiendaId=bbbbbbbb-bbbb-4bbb-8bbb-bbbbbbbbbbbb, zona="Zona Norte",
  monedaId=cccccccc-cccc-4ccc-8ccc-cccccccccccc,
  productoId=aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa (único ID de producto
garantizado
  por el seed fijo; el resto de productos se generan con UUID aleatorio
en tiempo
  de seed, así que no son predecibles).
Requerimientos del script:
* Debe permitir configurar por argumentos CLI:
   - --endpoint (GET o POST)
   - --users (concurrencia objetivo, es decir cuántos "usuarios
virtuales" en paralelo)
   - --ramp-up (segundos para ir levantando esos --users de forma
escalonada,
     no todos de golpe)
   - --duration (segundos que se sostiene la concurrencia máxima una
vez
     terminado el ramp-up)
* Modelo de carga: cada "usuario virtual" debe mandar peticiones EN
BUCLE
  CONTINUO (sin tiempo de espera entre una y otra) desde que se activa
(según
  el ramp-up) hasta que termine ramp-up + duration. Esto es para
estresar el
  backend de forma sostenida y encontrar el punto de inflexión real, no
para
  simular un patrón de "pensamiento humano" con pausas.
* Para POST, arma un body de "pedido grande" con más de 20 ítems
  (productoId, cantidad, precioUnitario, descuento, monedaId), usando
un
  identificador único (UUID) en cada request para no violar la
restricción
```

```
  de unicidad del campo `identificador`. Antes de arrancar la prueba,
intenta
  obtener productoId reales llamando GET /logistics/productos; si esa
llamada
  falla o no hay suficientes productos, cae en el productoId fijo del
seed
  repetido para los 20+ ítems.
* Debe registrar por cada request: timestamp (ISO), status_code,
latency_ms,
  y un campo error (vacío si fue exitosa; describe si fue timeout,
error de
  conexión, o código HTTP >= 400).
* Al final debe calcular e imprimir en consola:
   - total de requests enviadas
   - throughput real (req/s), calculado sobre el tiempo total real de
     ejecución (no el nominal)
   - latencia promedio, p95 y p99
   - Error % (status >= 400 + timeouts + errores de conexión, todos
cuentan
     como error)
* Debe exportar los resultados a un CSV llamado results_get.csv o
  results_post.csv según el endpoint probado, con columnas:
  timestamp_iso,status_code,latency_ms,error

Restricciones:
- Usa solo la librería estándar de Python, más EXACTAMENTE una librería
  externa: httpx, con async/await (asyncio).
- Maneja explícitamente timeouts (httpx.TimeoutException) y errores de
  conexión (httpx.RequestError), regístralos como error, no dejes que
  tumben el script completo.
- No generes gráficas (matplotlib) — solo el CSV, las voy a graficar yo
  manualmente en Excel para el informe.
- Ajusta los límites de conexión del cliente httpx (httpx.Limits) para
que
  soporte hasta 18000 usuarios concurrentes sin quedarse corto de
  conexiones disponibles en el pool del propio cliente.

Incluye instrucciones de ejecución con ejemplos, por ejemplo:
python scripts/load_test.py --endpoint GET --users 1500 --ramp-up 75 --
duration 60
python scripts/load_test.py --endpoint POST --users 3000 --ramp-up 100
--duration 60
```